

Capítulo 1

Lenguajes Regulares

1.1 Introducción

La teoría de la computación es el estudio matemático de las computadoras, sus modelos y capacidades. Para ello necesitamos desarrollar un modelo de computadora y un modelo de los datos que maneja. En lugar de hablar de computadoras, hablaremos de un dispositivo más general, denominado máquina o *autómata*, que es capaz de responder de acuerdo a un estímulo o dato de entrada. Cada autómata será diseñado para un propósito específico y reconocerá los datos de entrada codificados como una cadena de símbolos, pertenecientes a un determinado alfabeto básico.

La teoría de lenguajes formales estudia los conjuntos de cadenas de símbolos, llamados *lenguajes* y los distintos mecanismos para generarlos y reconocerlos. Un mecanismo para decidir si una determinada cadena de símbolos pertenece a un lenguaje es un autómata y un mecanismo para generar símbolos de un lenguaje es una *gramática*.

Los lenguajes naturales, es decir las lenguas humanas, se diferencian de los lenguajes formales, en que estos últimos son sistemas matemáticos. La teoría de los lenguajes formales incluye varios temas centrales: el *reconocimiento*, es decir cómo verificar que una cadena pertenece al lenguaje; la *generación* y la *representación* finita de los lenguajes.

El propósito del curso es comenzar el estudio de estos conceptos, sobre todo en su relación con las ciencias de la computación. En particular, en este capítulo estudiaremos denominados “lenguajes regulares”. Lo haremos desde los aspectos anteriormente mencionados, es decir consideraremos y un tipo de máquina abstracta que puede reconocerlos, un tipo de gramática que permite generarlos y un tipo de expresiones que permite denotarlos.

1.2 Alfabetos y Lenguajes

DEFINICIÓN 1.2.1 Diremos que un conjunto finito Σ es un *alfabeto* si $\Sigma \neq \emptyset$ y para todo $x \in \Sigma$, x es un símbolo indivisible (no está compuesto por subsímbolos). ■

EJEMPLO 1.2.1 $\Sigma = \{0, 1\}$, $\Sigma = \{a \dots z\}$, $\Sigma = \{0 \dots 9\}$ y $\Sigma = \{a, b\}$ son alfabetos,

$\Sigma = \{00, 10\}$ y $\Sigma = \{*, ||, ++\}$ no son alfabetos pues algunos símbolos están contruidos a partir de otros, es decir son divisibles.

□

DEFINICIÓN 1.2.2 Se dice que w es una *palabra* o *cadena* sobre un alfabeto Σ , si w es una secuencia finita de símbolos de Σ . ■

EJEMPLO 1.2.2 Si $\Sigma = \{0, 1\}$ es un alfabeto 0, 00, 000000000, 10, 1010101 son cadenas sobre el alfabeto Σ pero a , $0a$, $0000@0000$ y $101010z101010$ no lo son porque incluyen símbolos que no pertenecen a Σ . □

1.2.1 Operaciones sobre cadenas

Notaremos mediante Σ^* al conjunto de todas las cadenas formadas por elementos de un alfabeto Σ . Esto incluye la cadena de longitud 0 o cadena nula, representada mediante el símbolo λ .

EJEMPLO 1.2.3 Sea $\Sigma = \{0, 1\}$, $\Sigma^* = \{\lambda, 0, 1, 00, 01, 10, 11, 000, 001, 010, \dots\}$. Sea $\Sigma = \{a, \dots, z\}$, el alfabeto occidental utilizado en el lenguaje Español, $\Sigma^* = \{\lambda, a, zzzzzzzzzz, alfabeto, otebafla, esto es otra cadena, \dots\}$. □

Típicamente una palabra $w \in \Sigma^*$ se escribe como: $w = a_1 a_2 \dots a_k$ donde $a_i \in \Sigma$, $0 \leq i \leq k$. Si $k = 0$, la palabra w se denomina *palabra nula* o *palabra vacía* y se nota con λ . En tal caso k es llamada la *longitud* de w . Mas formalmente, consideraremos una función $long : \Sigma^* \rightarrow \mathbb{N}$ tal que $long(w) = k$ es el número de símbolos de Σ presentes en w , contando símbolos repetidos.

DEFINICIÓN 1.2.3 La *concatenación de dos cadenas* es una operación binaria $\bullet : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$ tal que, dadas dos cadenas x e y definidas sobre Σ , $x \bullet y = xy = w$ tal que $long(w) = long(x) + long(y)$ y w es tal que $w_i = x_i$ para $i = 1 \dots long(x)$ y $w_{long(x)+j} = y_j$ para $j = 1 \dots long(y)$. ■

Informalmente, la operación de concatenación “pega” dos cadenas formando una nueva. El símbolo de concatenación será omitido mas adelante cuando el lector ya esté familiarizado con la operación.

EJEMPLO 1.2.4 Sea $\Sigma = \{0, 1\}$ un alfabeto y $x = 0101$ e $y = 110$ dos cadenas del alfabeto. Entonces $x \bullet y = 0101110$ mientras que $y \bullet x = 1100101$. □

Observación: pueden demostrarse las siguientes propiedades de la operación de concatenación. Sea Σ un alfabeto,

1. La concatenación es asociativa, es decir para todo $x, y, z \in \Sigma^*$, $x \bullet (y \bullet z) = (x \bullet y) \bullet z$.
2. $[\Sigma^*, \bullet]$ es un monoide bajo concatenación y λ es el elemento identidad; es decir para todo $x \in \Sigma^*$, $x \bullet \lambda = \lambda \bullet x = x$.

DEFINICIÓN 1.2.4 Sea una cadena $w \in \Sigma^*$ y un número natural i , la *potencia i -ésima de la cadena w* , notada w^i , se define recursivamente como:

$$w^0 = \lambda$$

$$w^{(i+1)} = w \bullet w^i, \text{ para todo } i \geq 0$$

EJEMPLO 1.2.5 Sean $\Sigma = \{0, 1\}$ y $w = 01 \in \Sigma^*$, entonces $w^1 = 01, w^2 = 0101, \dots$ y así sucesivamente. □

DEFINICIÓN 1.2.5 Sea $w \in \Sigma^*$ tal que $w = a_1 a_2 \dots a_{n-1} a_n$, el reverso de la cadena w , $\overleftarrow{w}$, es la cadena $w = a_n a_{n-1} \dots a_2 a_1$. Podemos también definirla recursivamente mediante:

$\overleftarrow{w} = \lambda$, si $\text{long}(w) = 0$ (es decir, $w = \lambda$)

$\overleftarrow{w} = a \bullet \overleftarrow{u}$, si $\text{long}(w) = n + 1 > 0$ y $w = u \bullet a$ para $a \in \Sigma$ y $u \in \Sigma^*$ ■

EJEMPLO 1.2.6 Sea $w = \text{reverso}$, entonces $\overleftarrow{w} = \text{osrever}$. Sea $w = \text{amar}$, entonces $\overleftarrow{w} = \text{rama}$. □

Ahora a partir de las cadenas individuales definiremos los conjuntos de cadenas.

DEFINICIÓN 1.2.6 Se denomina *lenguaje L definido sobre Σ* , a todo subconjunto de Σ^* . ■

En particular Σ^* , Σ , $\{\lambda\}$ y $\emptyset$ son lenguajes que trivialmente respetan la definición. Los lenguajes son simplemente un tipo especial de conjuntos, entonces podemos especificar un lenguaje finito listando todos sus elementos, es decir sus cadenas.

EJEMPLO 1.2.7 Sea $\Sigma = \{a..z\}$ y $L \subseteq \Sigma^*$, $L = \{\text{ala}, \text{oso}, \text{planta}\}$ es un lenguaje. □

Algunos lenguajes de interés son infinitos, por lo tanto no pueden ser especificados por extensión sino que deben ser especificados usando alguna propiedad. Mas adelante se estudiará la forma de expresar un lenguaje infinito mediante una secuencia finita de símbolos.

1.2.2 Operaciones sobre Lenguajes

Ya que los lenguajes son conjuntos, sobre éstos son válidas las operaciones habituales de Unión, Intersección, Diferencia y Complemento. Además se definen las siguientes operaciones, propias de los lenguajes:

DEFINICIÓN 1.2.7 Sean L_1 y L_2 dos lenguajes definidos sobre Σ , su *concatenación L* es obtenida mediante $L = L_1 \bullet L_2 = \{w | w = x \bullet y \text{ para algún } x \in L_1 \text{ e } y \in L_2\}$. ■

Nota: dado que la operación de concatenación de lenguajes es una extensión de la operación de concatenación de cadenas, utilizaremos el mismo símbolo para ambas.

DEFINICIÓN 1.2.8 Sea $L \subseteq \Sigma^*$, definiremos *la potencia de lenguajes* como

$$\begin{aligned} L^0 &= \{\lambda\} \\ L^{(i+1)} &= L \bullet L^i, \text{ para todo } i \geq 0 \end{aligned}$$

DEFINICIÓN 1.2.9 Sea L un lenguaje, definimos la operación *Estrella de Kleene (o Clausura de Kleene)*, notada L^* , como

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Es decir, la estrella de Kleene es el conjunto de cadenas obtenidas por concatenación de cero o más cadenas de L . Entonces $L^* = \{w \in \Sigma^* | w = w_1 \bullet w_2 \bullet w_3 \dots \bullet w_k \text{ para } k \geq 0 \text{ y } w_1, w_2, \dots, w_k \in L\}$ es un nuevo lenguaje formado a partir de L .

EJEMPLO 1.2.8 Sea $L = \{a, ab, aaab\}$ entonces la cadena $aabaaab \in L^*$ pues puede formarse por la concatenación $a \bullet ab \bullet aaab$. □

Notaremos con L^+ al conjunto $L^+ = \bigcup_{i=1}^{\infty} L^i$, es decir $L^+ = LL^*$, esto es $\lambda \in L^+$ solo si $\lambda \in L$. En el caso de los alfabetos, $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma \bullet \Sigma^* = \Sigma^* - \{\lambda\}$. Esta última igualdad no se aplica a los lenguajes pues λ puede pertenecer al mismo.

LEMA 1.2.1 Sea Σ un alfabeto, para todo $L \subseteq \Sigma^*$, se verifica lo siguiente:

1. $\lambda \in L^*$
2. $\lambda \in L^+$ si y solo si $\lambda \in L$
3. $\lambda \notin \Sigma^+$

Demostración 1 queda como ejercicio para el lector.

☐

1.3 Autómatas Finitos

Los *autómatas finitos*, también denominados *máquinas de estados finitos* o simplemente *autómatas*, son entes o máquinas abstractas que estudiaremos desde dos puntos de vista: como dispositivos capaces de determinar la pertenencia de una cadena de símbolos a un lenguaje determinado y como traductores de una cadena en otra. En ambos casos los autómatas están computando una función, la primera emite una respuesta de tipo si-no y es frecuentemente referida en la jerga computacional como una “función booleana”.

Podemos imaginar un autómata finito como una máquina formada por un conjunto de estados, capaz de leer símbolos de una cinta. Operacionalmente funciona partiendo de un estado y cambiando de estado según el símbolo leído. Prosigue de esta forma hasta haber leído todos los símbolos de la cinta. Un estado puede ser pensado como una secuencia de valores que lo definen unívocamente. En tal sentido, puede pensarse que los estados en que puede encontrarse una computadora real está dado por los valores que tiene en su memoria y demás registros que la componen.

Un autómata finito consta entonces, de una cinta en la que se escribe una cadena de entrada, una memoria que recuerda el estado actual y una cabeza lecto-escritora que luego de leer un símbolo se mueve hacia la derecha y es capaz de decidir dado el estado corriente y el símbolo leído cual es el próximo estado. Esta descripción es sintetizada en la figura 1.1.

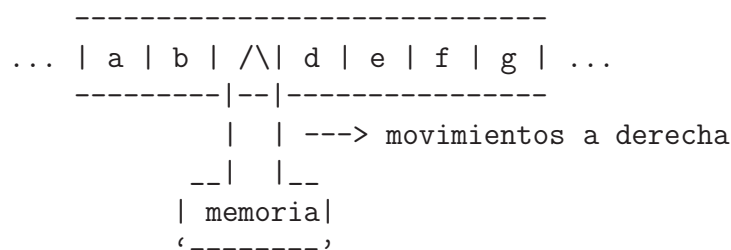


Figura 1.1: Esquema de un autómata finito

La memoria central del autómata queda determinada al momento del diseño, ya que el autómata “recuerda” a través de sus estados el conjunto de situaciones que puede tratar. Este conjunto de estados es finito.

Entre los estados siempre podemos distinguir tres categorías, no necesariamente disjuntas: el estado inicial, los estados finales o aceptadores y los estados intermedios. Si una máquina termina de leer todos los símbolos de una cinta y queda en un estado final, decimos que la

cadena es *aceptada* por el autómata. En caso contrario, decimos que la cadena es *rechazada* por el autómata.

Un autómata es construido para que reconozca las cadenas pertenecientes a un lenguaje L . Este lenguaje se define en base a un alfabeto y a un conjunto de propiedades. El autómata deberá estar construido respetando estas propiedades y así podrá reconocer al lenguaje L . Ya que los estados le permiten al autómata “recordar” situaciones que se desean diferenciar, es crucial en su diseño saber distinguir el conjunto de situaciones mínimas que se desea controlar.

1.3.1 Autómatas Finitos Reconocedores

Comenzaremos con el estudio de los autómatas finitos reconocedores. Informalmente, un autómata finito reconocedor es un dispositivo cuyo objetivo es reconocer un lenguaje. Reconocer un lenguaje significa que, para toda cadena definida sobre el alfabeto del lenguaje, se puede decidir si dicha cadena pertenece o no al mismo. Para ello, el autómata, de acuerdo a la entrada y a la evolución de sus estados, termina en un estado dado. Si este estado es aceptador, entonces la cadena es un elemento del lenguaje.

Hemos visto de nuestra explicación intuitiva que necesitamos disponer de un alfabeto de entrada, de un conjunto de estados aceptadores y de una función que nos permita hacer la evolución del autómata o *transición*. Damos a continuación la definición formal.

DEFINICIÓN 1.3.1 Un *autómata finito reconocedor (AFR)* es una quintupla $M = (S, \Sigma, \delta, s_0, F)$ donde:

S es un conjunto finito de estados, $S \neq \emptyset$.

Σ es el alfabeto de entrada.

$\delta : S \times \Sigma \rightarrow S$ es la función de “próximo estado” o de “transición”.

s_0 es el estado inicial, $s_0 \in S$.

F es el conjunto de estados finales o aceptadores: $F \subseteq S$.

■

Observe que la función de transición δ es una función binaria: de acuerdo a un estado de S y un elemento de Σ , nos indica a que otro estado el autómata puede evolucionar.

En algunos textos pueden encontrarse definiciones ligeramente distintas aunque equivalentes. En vez de dar un conjunto de estados aceptadores F , es equivalente dar una función de salida $f_0 : S \rightarrow \{0, 1\}$, donde arbitrariamente elegimos 1 como valor representativo de que la cadena procesada es aceptada y 0 para indicar que es rechazada.

EJEMPLO 1.3.1 Sea el autómata finito que reconoce el lenguaje L de identificadores válidos de Pascal, sin tener en cuenta longitud máxima. Según la definición de identificador, es una sucesión de una letra seguida por letras o dígitos; por lo tanto nos interesa registrar dos situaciones. En la inicial, asegurar que lea una letra como primer símbolo; en el resto del proceso asegurar que solo hay letras o dígitos. Además si en el estado inicial lee un dígito la cadena no es válida y deberá rechazarse. Por lo tanto, nuestro AFR tendrá tres estados. El estado inicial, s_0 . El estado s_1 , representativo de que entró o consumió una letra. El estado s_2 , de error señalando que un identificador no puede comenzar con un dígito.

Luego, el autómata finito $M = (S, \Sigma, \delta, s_0, F)$ permite reconocer el lenguaje requerido definiendo:

$$S = \{s_0, s_1, s_2\}$$

$$\Sigma = \{a \dots z, 0 \dots 9\}$$

$$F = \{s_1\}$$

δ	“letra”	“dígito”
s_0	s_1	s_2
s_1	s_1	s_1
s_2	s_2	s_2

Si bien el alfabeto Σ indica en forma individual a cada letra y dígito, en la tabla de estados indicamos sólo una columna para las 28 letras y otra para los dígitos por simplicidad. También para simplificar la explicación omitimos otros símbolos. $\square$

EJEMPLO 1.3.2 Diseñar un AFR que acepte el lenguaje de las cadenas de símbolos pertenecientes al alfabeto $\{a, b\}$ y que contengan tres b 's consecutivas. Queremos que el AFR M reconozca las cadenas con cualquier cantidad de a y 3 b 's consecutivas; por lo tanto, estando en la situación inicial, si lee una a , el estado no se altera; si lee una b , tendremos otra situación: “se leyó una b ”, otra por haber leído dos b y otra por tres b . En la última situación debe aceptar y en cualquiera de las situaciones anteriores, si lee una a rompe la consecutividad, por lo tanto vuelve al estado inicial. Luego podemos organizar los estados para representar las siguientes situaciones:

s_0 es el estado inicial,

s_1 el estado de haber leído una b ,

s_2 el estado de haber leído dos b consecutivas,

s_3 el estado de haber leído tres b consecutivas y además es el estado aceptador.

Un autómata finito $M = (S, \Sigma, \delta, s_0, F)$ capaz de realizar este proceso puede ser definido mediante:

$$S = \{s_0, s_1, s_2, s_3\}$$

$$\Sigma = \{a, b\}$$

$$F = \{s_3\}$$

δ	a	b
s_0	s_0	s_1
s_1	s_0	s_2
s_2	s_0	s_3
s_3	s_3	s_3

Algunas cadenas aceptadas por el autómata son $aabbba$, bbb , $abbb$, mientras que $abbab$, $aaaaa$ y λ son rechazadas. $\square$

EJEMPLO 1.3.3 Encontrar un autómata con menos estado que resuelva el mismo problema. $\square$

1.3.2 Representación de Autómatas mediante Grafos Dirigidos

Un grafo dirigido es una terna (N, A, V) donde N es el conjunto de vértices o nodos, A es el conjunto de arcos y V es la función que asocia a cada arco $a \in A$ con un par ordenado de vértices. Aquí utilizaremos multidigraphos para representar autómatas finitos y los llamaremos *Diagrama de Estados*. Las convenciones a adoptar para tal fin serán las siguientes:

Existe un nodo $n \in N$ del grafo por cada estado $s \in S$.

Existe un arco $a \in A$ tal que $V(a) = (s_i, s_k)$ si y solo si $\delta(s_i, a) = s_k$, donde $s_i, s_k \in S, a \in \Sigma$.

Debe notarse que cada arco puede tener más de un rótulo.

El estado inicial será indicado con el símbolo “ $\rightarrow$ ” y los estados finales con un doble círculo.

EJEMPLO 1.3.4 retomemos el ejemplo del lenguaje de los identificadores. Los nodos del grafo son s_0, s_1 y s_2 . El estado inicial es s_0 y el aceptador s_1 . Los arcos estan rotulados con las letras “d” y “l” como abreviación de “digito” y “letra”. Es decir, representan a cualquier digito y letra del alfabeto en forma genérica. En la figura 1.2 proveemos un grafo que representa el autómata descripto.

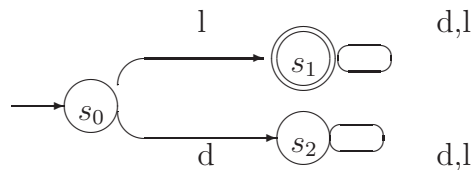


Figura 1.2: AF para reconocimiento de identificadores de Pascal.

□

EJEMPLO 1.3.5 Diseñar un autómata finito que reconozca el lenguaje de las cadenas de números definidas sobre el alfabeto $\Sigma = \{0, 1\}$ que representan los números binarios pares. Los números binarios pares terminan en 0. Por lo tanto nos interesa registrar cuándo un número termina en 0. La definición del autómata es dada a continuación, y el grafo correspondiente es dado en la figura 1.3.

$$M = (S, \Sigma, \delta, s_0, F)$$

$$\Sigma = \{0, 1\}$$

$$F = \{s_0\}$$

$$S = \{s_0, s_1\}$$

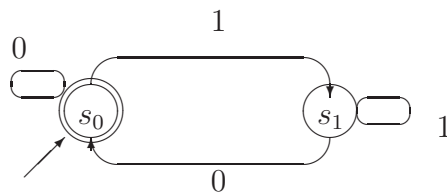


Figura 1.3: AF para reconocer numeros binarios pares.

□

1.3.3 Extensiones a la definición de Autómata Finito

Otro aspecto de la definición de autómata finito a tener en cuenta es la posibilidad de incorporar a la definición de la función de transición la posibilidad de usar λ como símbolo a ser procesado por el autómata. Si bien la posibilidad de permitir movimientos sin consumir símbolos puede parecer innecesaria, en breve se percibirá su utilidad teórica a través de su uso en demostraciones. Por otra parte es necesario tener en cuenta que este agregado no incrementa la potencia de los autómatas finitos, lo cual es considerado a través del próximo resultado.

LEMA 1.3.1 Dado un autómata finito que permite transiciones etiquetadas con λ , puede construirse otro autómata equivalente sin transiciones etiquetadas con λ . ■

Demostración 2 (esquema) Dado un AF M puede obtenerse otro M' equivalente y sin transiciones etiquetadas con λ a través de las siguientes transformaciones:

1. Agregar a M λ -transiciones de un estado s_i a otro s_j cuando en M se llegaba desde s_i a s_j por medio de mas de una λ -transición
2. poner como aceptadores todos aquellos estados de M que por λ permiten llegar a un estado aceptador
3. por cada par de transiciones $\delta(s_i, \lambda) = s_j, \delta(s_j, a) = s_k$ agregar $\delta(s_i, a) = s_k$
4. sacar del autómata obtenido hasta el momento las λ -transiciones $\delta(s_i, \lambda) = s_j$ para todo $s_i, s_j \in S$

□

Esto da lugar entonces a una reformulación de la definición de autómata finito incorporando la posibilidad de realizar transiciones sin consumir elementos en la cadena de entrada del AF.

DEFINICIÓN 1.3.2 Un *autómata finito reconocedor* (AFR) es una quintupla $M = (S, \Sigma, \delta, s_0, F)$ donde:

S es un conjunto finito de estados, $S \neq \emptyset$.

Σ es el alfabeto de entrada.

$\delta : S \times (\Sigma \cup \{\lambda\}) \rightarrow S$ es la función de próximo estado o de transición.

s_0 es el estado inicial, $s_0 \in S$.

F es el conjunto de estados finales o aceptadores: $F \subseteq S$. ■

Luego, en virtud de la equivalencia y de la utilidad antes mencionada, utilizaremos de aquí en más ésta última definición como la estándar en el curso.

Otra extensión a la función δ consiste en permitir su uso en una forma mas flexible. Esto puede lograrse extendiendo la definición de δ para que procese cadenas en vez de símbolos, donde el estado resultado es el que se obtiene de aplicar δ a partir del estado inicial y a cada

símbolo de la cadena. Luego en algunos casos consideraremos una definición de autómata finito en el cual δ será una función $\delta' : S \times \Sigma^* \rightarrow S$ tal que

$$\delta'(s_i, \lambda) = s_i$$

$$\delta'(s_i, ax) = \delta'(\delta(s_i, a), x) \text{ cuando } a \in \Sigma \text{ y } x \in \Sigma^*.$$

EJEMPLO 1.3.6 En el autómata finito del ejemplo 1.3.2 $\delta'(s_0, aabbba) = s_3$ pues:

$$\begin{aligned} \delta'(\delta(s_0, a), aabbba) &= \\ \delta'(\delta(\delta(s_0, a), a), bba) &= \\ \delta'(\delta(\delta(\delta(s_0, a), a), b), ba) &= \\ \delta'(\delta(\delta(\delta(\delta(s_0, a), a), b), b), a) &= \\ \delta'(\delta(\delta(\delta(\delta(\delta(s_0, a), a), b), b), b), \lambda) &= \\ \delta'(\delta(\delta(\delta(\delta(\delta(s_0, a), a), b), b), b), a) &= \\ \delta'(\delta(\delta(\delta(\delta(s_0, a), b), b), b), a) &= \\ \delta'(\delta(\delta(\delta(s_0, b), b), b), a) &= \\ \delta'(\delta(\delta(s_1, b), b), a) &= \\ \delta'(\delta(s_2, b), a) = \delta'(s_3, a) = \delta(s_3, a) &= s_3 \end{aligned}$$

□

Dado que δ y δ' no son contradictorias las usaremos indistintamente notándolas con δ .

EJEMPLO 1.3.7 Sea δ la función de transición de un autómata finito. Probar que para cadenas de entrada x e y cualesquiera:

$$\delta'(q, xy) = \delta'(\delta'(q, x), y)$$

Sugerencia: usar inducción sobre la longitud de x . ■

Pasaremos ahora a considerar una serie de conceptos tendientes a establecer más formalmente cuando puede decirse que una cadena es reconocida (o un lenguaje es reconocido) a través de un autómata finito.

DEFINICIÓN 1.3.3 Sea un autómata finito $M = (S, \Sigma, \delta, s_0, F)$, llamaremos *configuración* a cada par $(s, w) \in S \times \Sigma^*$. ■

Una configuración representa la situación global en que se encuentra M en un momento dado. En cada par de esta forma, s representa el estado actual y w la parte aún no leída de la cadena de entrada. La configuración inicial es (s_0, w) donde $s_0 \in S$ es el estado inicial y $w \in \Sigma^*$ es la cadena ha ser reconocida.

DEFINICIÓN 1.3.4 Una *transición* de M será representada por la relación binaria “ $\vdash$ ” entre configuraciones: $(s, aw) \vdash (s', w)$ sssi $\delta(s, a) = (s')$ donde $s, s' \in S$; $a \in \Sigma \cup \{\lambda\}$; $w \in \Sigma^*$. ■

Una transición indica la evolución del autómata. Si $a \neq \lambda$ determina que si M está en una configuración tal que el control está en un estado s y el símbolo corriente es a entonces M pasará a una configuración con estado s' y la cabeza lectora se ha movido a la derecha un lugar. Si en el lugar de a considerasemos λ entonces se asume que la cabeza lectora del autómata no avanza.

Notación: mediante $\vdash^+$ y $\vdash^*$ representaremos la clausura transitiva y la reflexo-transitiva de $\vdash$, respectivamente.

DEFINICIÓN 1.3.5 Se dice que el AFR $M = (S, \Sigma, \delta, s_0, F)$ *acepta (o reconoce) una cadena* $w \in \Sigma^*$ *sssi* $(s_0, w) \vdash^* (s_f, \lambda)$, donde $s_f \in F$ ■

Es decir si M , comenzando en el estado inicial s_0 , lee la cadena w de izquierda a derecha y termina en un estado final o aceptador.

DEFINICIÓN 1.3.6 Sea un AFR $M = (S, \Sigma, \delta, S_0, F)$, el *lenguaje reconocido por el autómata* M es $L(M) = \{w \in \Sigma^* | w \text{ es aceptada por } M\}$. ■

EJEMPLO 1.3.8 Retomando el caso del autómata para reconocer identificadores de Pascal, $(s_0, \text{variable}) \vdash (s_1, \text{variable}) \vdash^+ (s_1, \lambda)$ y entonces podemos decir que el autómata acepta el identificador *variable*. También $(s_0, 4\text{variable}) \vdash (s_2, \text{variable}) \vdash^+ (s_2, \lambda)$ y entonces el identificador *4variable* es rechazado por el autómata. Como en dicho autómata el único estado aceptador es s_1 solo las cadenas w que satisfacen $(s_0, w) \vdash^* (s_1, \lambda)$ son aceptadas y pertenecen al lenguaje aceptado por el autómata. □

1.3.4 Autómatas Finitos Traductores

Hasta ahora hemos tratado los autómatas finitos como reconocedores de cadenas que pertenecen a un lenguaje donde su aceptación o rechazo está dado por el estado de S en que termina la computación. Pensemos ahora si podría ser útil considerar autómatas finitos para computar funciones, donde la función de salida en lugar de ser la aceptación o rechazo de una cadena sea una función más compleja que mapea o *traduce* cadenas de entrada en cadenas de salida. Un ejemplo sencillo que requiere tal tipo de capacidad computacional se presenta cuando dada una cadena definida sobre un alfabeto, digamos $\Sigma = \{a, b\}$, deseamos contar el número de apariciones de una subcadena, por ejemplo *ab*. Aquí cuando utilizamos el término “contar”, no lo hacemos en el sentido estricto requiriendo que la salida del AF sea un número en notación decimal, sino en alguna otra notación más acorde con este dispositivo. Por ejemplo, la notación unaria o de números uno, donde se emiten tantos números uno como apariciones de *ab* aparezcan en la cadena.

Desde este punto de vista, estamos estudiando a los autómatas finitos como un modelo de computadora restringida, ya que el objetivo es realizar un cálculo a partir de una cadena. Para ello será necesario cambiar en la definición de M el conjunto de estados aceptadores por una función de salida f_0 ; donde f_0 está definida del conjunto de estados en el alfabeto de salida. Esto conduce a los denominados *Autómatas Finitos Traductores (AFT)*, de los cuales consideraremos dos clases, los AFT de Moore y los AFT de Mealy.

DEFINICIÓN 1.3.7 Un *autómata finito traductor de Moore* es una sextupla

$$M = (S, \Sigma, Z, \delta, s_0, f_0)$$

donde $f_0 : S \rightarrow Z$. Aquí Z es el alfabeto de salida y puede variar de acuerdo al problema planteado. Los demás elementos se interpretan como en el caso de un AFR. ■

La función de salida se indicará en la tabla de estados con una columna extra:

<i>estados</i>	<i>entradas</i>	<i>salida</i>
s_0	$\dots$	z_0
s_1	$\dots$	z_1
$\dots$	$\dots$	$\dots$

EJEMPLO 1.3.9 sea el problema de contar el número de apariciones de la subcadena ab en cadenas de $\{a, b\}^*$. Consideremos una solución del problema a través de la definición de un AFT de Moore. El autómata finito traductor, deberá reconocer dos situaciones: se acaba de leer una a , en cuyo caso no hay salida; se lee una b , luego de haber leído una a , en cuyo caso hay que emitir un “1”.

$$M = (S, \Sigma, Z, \delta, s_0, f_0)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$\Sigma = \{a, b\}$$

$$Z = \{1\}$$

δ	a	b	f_0
s_0	s_1	s_3	-
s_1	s_1	s_2	-
s_2	s_1	s_3	1
s_3	s_1	s_3	-

El AFT tiene cuatro estados, s_1 registra el hecho de haber leído una a y s_2 indica que se ha leído una b , luego de una sucesión de letras a , en particular luego de haber leído una a que es lo relevante para el problema planteado. Se utiliza el símbolo ‘-’ para indicar que no se emite salida. Luego solo se traduce cuando se encuentra la cadena ab y se traduce al caracter 1. $\square$

Ejercicio: ¿Puede dar un AFT equivalente con menos estados?.

DEFINICIÓN 1.3.8 Un *autómata finito traductor de Mealy* es una sextupla

$$M = (S, \Sigma, Z, \delta, s_0, f_0)$$

donde $f_0 : S \times \Sigma \rightarrow Z$ y los demas elementos se interpretan como en el caso de AFT de Moore. $\blacksquare$

EJEMPLO 1.3.10 Una solución del problema anterior utilizando un AFT de Mealy es la siguiente:

$$M = (S, \Sigma, Z, \delta, s_0, f_0)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$\Sigma = \{a, b\}$$

$$Z = \{1\}$$

δ	a	b
s_0	$s_1/-$	$s_3/-$
s_1	$s_1/-$	$s_2/1$
s_2	$s_1/-$	$s_3/-$
s_3	$s_1/-$	$s_3/-$

□

Ejercicio: desarrollar un AFT que compute la suma de dos números binarios. *Sugerencia:* Utilice como entrada un par de dígitos binarios (a, b) , donde a es el dígito i -ésimo del primer número y b es el dígito i -ésimo del segundo número.

1.3.5 Autómatas finitos no deterministas

Hasta el momento, todos los AF que hemos estudiado son tales que dado el estado corriente y un símbolo del alfabeto, la función de próximo estado estaba unívocamente determinada. A estos autómatas los denominaremos autómatas finitos deterministas (AFD). En esta sección analizaremos un tipo de autómata en que esto no sucede. A estos los denominaremos autómatas finitos no deterministas (AFND).

El concepto de no determinismo, si bien inicialmente no es intuitivo, es muy importante y especialmente útil en prueba de teoremas. Recordemos que la definición dada de autómata finito es determinista, pues $\delta(q_i, a) = q_j$ con $q_i, q_j \in S$ y $a \in \Sigma$. Es decir, dados un estado y un símbolo de entrada, el próximo estado queda determinado unívocamente.

La característica de no determinismo le da al autómata finito la posibilidad de cambiar de estados en una forma que está solo parcialmente determinada por el símbolo corriente y el estado corriente, pues también depende de una elección. Si bien el concepto de no determinismo aparece como una extensión de determinismo y en muchos casos simplifica la descripción de los autómatas, probaremos que es una característica no esencial ya que para todo autómata finito no determinista existe uno equivalente determinista.

DEFINICIÓN 1.3.9 Un *autómata finito reconocedor no determinista (AFND)* es definido mediante una quintupla $M = (S, \Sigma, \delta, s_0, F)$, donde

S , es un conjunto finito de estados tal que $S \neq \emptyset$.

Σ es el alfabeto de entrada.

$\delta : S \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(S)$.

$s_0 \in S$ es el estado inicial.

$F \subseteq S$ es el conjunto de estados finales.

■

Debe notarse que la diferencia entre ambas definiciones es que $\delta(s_i, a)$ es igual a un subconjunto de estados (posiblemente vacío) en lugar de un estado. La interpretación de $\delta(s_i, a) = \{s_j, s_{j+1}, \dots, s_m\}$ es que el autómata finito M en el estado s_i , con la entrada a , debe elegir alguno de los elementos del conjunto $\{s_j, s_{j+1}, \dots, s_m\}$ como próximo estado.

EJEMPLO 1.3.11 Diseñar un autómata finito que acepte el lenguaje de las cadenas formadas por concatenación de 0 o más cadenas de ab o aba (sin importar el orden).

En este caso un autómata finito no determinista es más conveniente que un autómata finito determinista, pues si bien se trata de un lenguaje simple, el autómata finito determinista que lo reconoce es:

$$M' = (S', \Sigma', \delta', s'_0, F')$$

$$S' = \{s'_0, s'_1, s'_2, s'_3, s'_4\}$$

$$\Sigma' = \{a, b\}$$

$$F' = \{s'_0, s'_2, s'_3\}$$

δ'	a	b
s'_0	s'_1	s'_4
s'_1	s'_4	s'_2
s'_2	s'_3	s'_4
s'_3	s'_1	s'_2
s'_4	s'_4	s'_4

Aún dibujando el diagrama de estados toma un cierto tiempo verificar que de cada nodo parten dos arcos, uno para a y otro para b , y si bien es válido suponer que se puede encontrar un autómata finito determinista más simple, se puede probar formalmente que es imposible. La prueba formal aludida será mas clara para el lector al final del capítulo.

Sin embargo el lenguaje es aceptado por el siguiente autómata finito no determinista que es más simple:

$$M = (S, \Sigma, \delta, s_0, F)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$F = \{s_0\}$$

δ	a	b
s_0	$\{s_1\}$	$\{s_3\}$
s_1	$\{s_3\}$	$\{s_0, s_2\}$
s_2	$\{s_0\}$	$\{s_3\}$
s_3	$\{s_3\}$	$\{s_3\}$

¿Qué particularidades tiene? Este autómata finito en el estado s_1 , leyendo una b tiene dos posibles próximos estados. En otros casos es conveniente rotular los arcos o agregar a las entradas ϵ , que quiere decir que no consume símbolo de la cadena de entrada. $\square$

Para los autómatas finitos no deterministas diremos que una cadena w es aceptada si existe algún camino del estado inicial s_0 a un estado aceptador q , siguiendo la función δ , sin importar que haya posibilidades para descartar. Mas formalmente:

DEFINICIÓN 1.3.10 Se dice que una cadena w es *aceptada* por un autómata finito no Determinista $M = (S, \Sigma, \delta, s_0, F)$ si existe un estado $p \in F$ tal que $\delta(s_0, w) \supseteq \{p\}$. $\blacksquare$

EJEMPLO 1.3.12 En el autómata del ejemplo anterior $w = aba$ es aceptada ya que existe un camino s_0, s_1, s_2, s_0 , que termina en un estado aceptador ($s_0 \in F$). En cambio, si hubiera optado por s_0, s_1, s_0, s_1 habría finalizado en un estado no aceptador, pero eso no es tenido en cuenta ya que lo que importa es que exista una secuencia de pasos que permitan aceptar la cadena en cuestión. $\square$

Ahora formalizaremos la idea intuitivamente planteada, que M y M' son equivalentes.

Ejercicio: Al terminar este capítulo intente probar que no existe otro autómata finito determinista que reconozca al lenguaje del ejercicio anterior y sea más simple.

1.3.6 Equivalencia entre AFD y AFND

TEOREMA 1.3.1 Sea L un lenguaje aceptado por un autómata finito no determinista, entonces existe un autómata finito determinista que también acepta L . $\blacksquare$

Demostración 3 Sea $M = (S, \Sigma, \delta, s_0, F)$ un autómata finito no determinista que acepta L . Para probar el teorema debemos probar: a) que podemos construir M' determinista, a partir de M y b) que M' acepta el mismo lenguaje que M .

Construimos un autómata finito determinista $M' = (S', \Sigma, \delta', s'_0, F')$ tal que :

$S' \subseteq \mathcal{P}(S)$. Los estados de M' serán identificados con subconjuntos del conjunto de estados de M . *Notación:* un elemento de S' se notará como $\langle s_1, s_2, \dots, s_i \rangle$ donde $\{s_1, s_2, \dots, s_i\} \subseteq S$.

$F' = \{s'_i \in S' | s'_i = \langle \dots s_j \dots \rangle \text{ y } s_j \in F\}$. F' es el conjunto de todos los estados de S' que contengan por lo menos un elemento de F (es decir, un estado aceptador de M).

Σ es el alfabeto de entrada.

$s'_0 = \langle s_0 \rangle = \{s_0\}$. s'_0 será el conjunto que contiene como único estado al inicial de M .

la función δ' es la extensión de δ a conjuntos:

$$\delta'(\langle s_1, s_2, \dots, s_i \rangle, a) = \langle p_1, p_2, \dots, p_j \rangle \text{ si y solo si } \delta(\{s_1, s_2, \dots, s_i\}, a) = \{p_1, p_2, \dots, p_j\}$$

esto es, δ' aplicada a un elemento ζ de S' se computa aplicando δ a cada estado de S representado en ζ (como δ' se aplica a un conjunto de estados, el resultado es el conjunto de los estados que surgen de aplicar δ a cada uno de los estados, $\delta(\{s_1, \dots, s_j\}, a) = \bigcup_{i=1}^j \delta(s_i, a)$). Es decir, si $\zeta = \langle s_1, s_2, \dots, s_n \rangle$, aplicando δ a cada s_j , $1 \leq j \leq n$, y tomando la unión obtenemos los estados de llegada $p_1, \dots, p_k$. Este nuevo conjunto tiene un representante en S' , $\langle p_1 \dots p_k \rangle$, que es el resultado de $\delta'(\zeta, a)$.

Observación: se debe tener en cuenta que, mientras que el resultado de la función δ es un conjunto de estados de M (por lo cual M es no determinista), el resultado de δ' , en cambio, es un único estado de M' , denotado $\langle q_1, \dots, q_j \rangle$ (por lo cual M' es determinista). La diferencia está en que, en un caso, el conjunto de estados es un subconjunto de $\mathcal{P}(S)$, mientras que en el otro, es S .

Ya definimos M' a partir de M . Sólo falta probar que $\delta'(s'_0, x) = \langle q_1, q_2 \dots q_i \rangle$ si y sólo si $\delta(s_0, x) = \{q_1, q_2, \dots, q_i\}$ para toda cadena x . Lo probaremos por inducción sobre la longitud de la cadena x .

Base de inducción: Si $x = \lambda$ entonces la afirmación es correcta ya que $s'_0 = \langle s_0 \rangle = \{s_0\}$.

Paso de inducción: Supongamos que es válido para $\text{long}(x) = l$ es decir $\delta'(s'_0, x) = \langle p_1, \dots, p_n \rangle$ si y solo si $\delta(s_0, x) = \{p_1 \dots p_n\}$, sea $a \in \Sigma$, debemos probar que vale para $\text{long}(xa) = l + 1$. Sabemos por definición de δ' que

$$\delta'(s'_0, xa) = \delta'(\delta'(s'_0, x), a) \quad (1.1)$$

por hipótesis de inducción, tenemos:

$$\delta'(s'_0, x) = \langle p_1 \dots p_n \rangle \text{ si y solo si } \delta(s_0, x) = \{p_1, \dots, p_n\}$$

reemplazando en 1.1 y por definición de δ' , nos queda:

$$\delta'(\langle p_1 \dots p_n \rangle, a) = \langle q_1 \dots q_k \rangle \text{ si y solo si } \delta(\{p_1, \dots, p_n\}, a) = \{q_1, \dots, q_k\}$$

Entonces

$$\delta'(s'_0, xa) = \langle q_1 \dots q_k \rangle \text{ si y solo si } \delta(s_0, xa) = \{q_1, \dots, q_k\}$$

Nota: tener en cuenta que $\delta'(s'_0, x)$ está en F' (es un estadoaceptador de M') cuando $\delta(s_0, x)$ tiene un estado ζ perteneciente a F , por lo tanto, una cadena será aceptada por M' si y sólo si es aceptada por M .

Por lo tanto concluimos que los autómatas finitos deterministas y los autómatas finitos no deterministas aceptan el mismo conjunto de lenguajes. $\square$

EJEMPLO 1.3.13 Sea un AFND $M = (S, \Sigma, \delta, s_0, F)$ donde:

$$S = \{s_0, s_1, s_2\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{s_1\}$$

δ	0	1
s_0	$\{s_0, s_1\}$	$\{s_1\}$
s_1	$\{s_2\}$	$\{s_0, s_1\}$
s_2	$\{s_2\}$	$\{s_2\}$

obtener un autómata finito determinista M' que reconozca el mismo lenguaje utilizando el teorema anterior. Construimos un AFD $M' = (S', \Sigma', \delta', \langle s_0 \rangle, F')$ donde

$$S' = \{\langle s_0 \rangle, \langle s_1 \rangle, \langle s_2 \rangle, \langle s_0, s_1 \rangle, \langle s_0, s_1, s_2 \rangle\}$$

$$\Sigma' = \Sigma$$

$$F' = \{\langle s_1 \rangle, \langle s_0, s_1 \rangle, \langle s_0, s_1, s_2 \rangle\}$$

δ'	0	1
$\langle s_0 \rangle$	$\langle s_0, s_1 \rangle$	$\langle s_1 \rangle$
$\langle s_1 \rangle$	$\langle s_2 \rangle$	$\langle s_0, s_1 \rangle$
$\langle s_2 \rangle$	$\langle s_2 \rangle$	$\langle s_2 \rangle$

Nos falta definir $\delta'(\langle s_0, s_1 \rangle, 0)$, $\delta'(\langle s_0, s_1 \rangle, 1)$, $\delta'(\langle s_0, s_1, s_2 \rangle, 0)$ y $\delta'(\langle s_0, s_1, s_2 \rangle, 1)$:

$$\delta'(\langle s_0, s_1 \rangle, 0) = \delta(\{s_0, s_1\}, 0) = \delta(s_0, 0) \cup \delta(s_1, 0) = \{s_0, s_1\} \cup \{s_2\} = \langle s_0, s_1, s_2 \rangle$$

$$\delta'(\langle s_0, s_1 \rangle, 1) = \delta(\{s_0, s_1\}, 1) = \delta(s_0, 1) \cup \delta(s_1, 1) = \{s_1\} \cup \{s_0, s_1\} = \langle s_0, s_1 \rangle$$

$$\delta'(\langle s_0, s_1, s_2 \rangle, 0) = \delta(\{s_0, s_1, s_2\}, 0) = \delta(s_0, 0) \cup \delta(s_1, 0) \cup \delta(s_2, 0) = \dots = \langle s_0, s_1, s_2 \rangle$$

$$\delta'(\langle s_0, s_1, s_2 \rangle, 1) = \delta(\{s_0, s_1, s_2\}, 1) = \delta(s_0, 1) \cup \delta(s_1, 1) \cup \delta(s_2, 1) = \dots = \langle s_0, s_1, s_2 \rangle$$

Entonces δ' completa es:

δ'	0	1
$\langle s_0 \rangle$	$\langle s_0, s_1 \rangle$	$\langle s_1 \rangle$
$\langle s_1 \rangle$	$\langle s_2 \rangle$	$\langle s_0, s_1 \rangle$
$\langle s_2 \rangle$	$\langle s_2 \rangle$	$\langle s_2 \rangle$
$\langle s_0, s_1 \rangle$	$\langle s_0, s_1, s_2 \rangle$	$\langle s_0, s_1 \rangle$
$\langle s_0, s_1, s_2 \rangle$	$\langle s_0, s_1, s_2 \rangle$	$\langle s_0, s_1, s_2 \rangle$

□

Ejercicio: Aplicar el método sugerido en la demostración para obtener el AFD dado en la sección anterior, a partir del no determinista previo.

Ejercicio: Dado el autómata finito no determinista que acepta el lenguaje de cadenas w tales que w posee o dos 0's o dos 1's consecutivos, utilizar el teorema para obtener el autómata finito determinista.

Al igual que en el caso de autómatas finitos deterministas, es posible definir los conceptos de *configuración* y *transición* tendientes a definir formalmente el significado de los terminos *cadena reconocida* y *lenguaje reconocido* considerando autómatas no deterministas. Estas definiciones surgen facilmente como ampliaciones de las dadas para el caso de AFD y son dejadas como ejercicio para el lector.

1.4 Homomorfismos sobre Autómatas Finitos

Bajo la consideración que el lector conoce las nociones de homomorfismos e isomorfismos a partir de un curso previo, veremos aquí cómo dichos conceptos son útiles para el estudio de los autómatas finitos. El objetivo será aprender a establecer un homomorfismo entre dos AFD M y M' , lo cual a su vez nos permitirá definir un método para simplificar autómatas. La idea es que dado un AF M , podamos estudiar si existe un AF M' que realice lo mismo que M pero tenga menos estados.

El lector debe advertir que en la definición siguiente utilizaremos una función $f_0 : S \rightarrow \{0, 1\}$. La misma reemplaza al conjunto de estados aceptadores utilizados inicialmente, para indicar los estados aceptadores en un autómata finito reconocedor donde $f_0(s) = 0$ para todo $s \notin F$ y $f_0(s) = 1$ para todo $s \in F$. La motivación de este cambio momentáneo es doble. En primer lugar, es una notación que el lector frecuentemente puede encontrar en

los libros de texto y es conveniente que conozca. Por otra parte, facilitará la resolución de los ejercicios planteados al final de esta sección y la siguiente.

Sean $M = (S, \Sigma, s_0, \delta, f_0)$ y $M' = (S', \Sigma, s'_0, \delta', f'_0)$ dos AFD reconocedores. Un *homomorfismo* g del autómata finito M al autómata finito M' es una función $g : S \rightarrow S'$ tal que para cualquier $i \in \Sigma$ y $s \in S$:

$$g(s_0) = s'_0 \quad (1.2)$$

$$g(\delta(s, i)) = \delta'(g(s), i) \quad (1.3)$$

$$f_0(s) = f'_0(g(s)) \quad (1.4)$$

Es decir el homomorfismo g conserva la función de transición o de próximo estado y la función de salida. La función g asocia estados de una máquina con estados de otra. La ecuación 1.3 dice que es equivalente aplicar el homomorfismo al estado resultante de aplicar la función δ al estado s y entrada i , o aplicar la δ' del autómata finito homomorfo a la imagen homomorfa del estado s y la entrada i (ver figura 1.4).

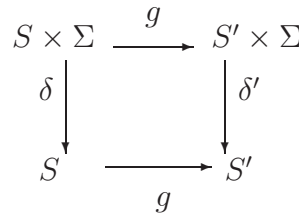


Figura 1.4: Diagrama Conmutativo del homomorfismo sobre AF

La ecuación 1.4 obliga a que los estados mapeados por g se comporten igual que en M respecto de las cadenas que son aceptadas o rechazadas, asegurando así que reconozcan el mismo lenguaje. La ecuación 1.2 asegura que ambos tengan un inicio coordinado y junto con 1.4, a que los autómatas sean equivalentes en su efecto, es decir reconozcan el mismo lenguaje. A modo de simplificación de las presentaciones, omitiremos la consideración de este último aspecto en el futuro. Asumiremos que se satisface dicho requerimiento sobre los estados iniciales de los autómatas involucrados en los homomorfismos analizados.

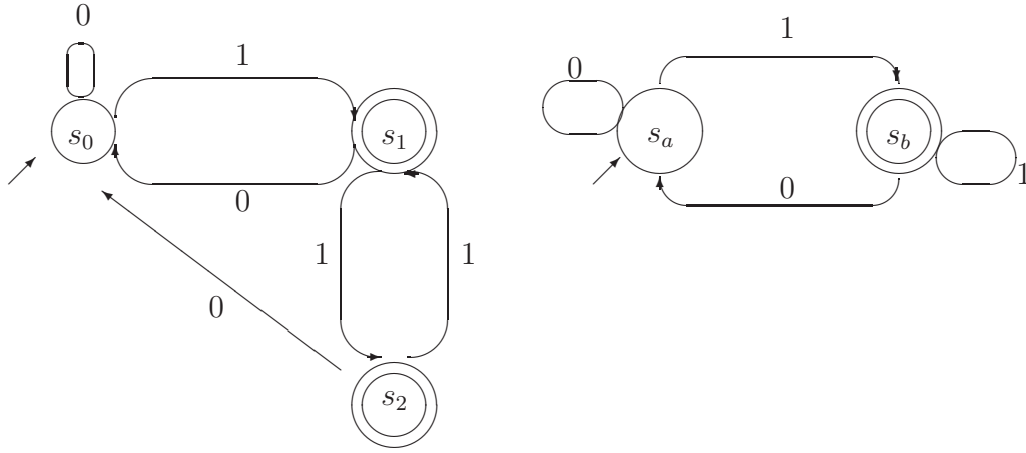
EJEMPLO 1.4.1 Dados los siguientes autómatas finitos:

$$\begin{array}{ll}
 M = (S, \Sigma, s_0, \delta, f_0) & M' = (S', \Sigma, s'_0, \delta', f'_0) \\
 S = \{s_0, s_1, s_2\} & S' = \{s_a, s_b\} \\
 \Sigma = \{0, 1\} &
 \end{array}$$

$$f_0 =_{def} \begin{cases} f(s_0) = 0 \\ f(s_1) = f(s_2) = 1 \end{cases} \quad f'_0 =_{def} \begin{cases} f'(s_a) = 0 \\ f'(s_b) = 1 \end{cases}$$

δ	0	1
s_0	s_0	s_1
s_1	s_0	s_2
s_2	s_0	s_1

δ'	0	1
s_a	s_a	s_b
s_b	s_a	s_b



definimos una función $g : S \rightarrow S'$

$$g =_{def} \begin{cases} g(s_0) = s_a \\ g(s_1) = s_b \\ g(s_2) = s_b \end{cases}$$

Demostremos que g define un homomorfismo de M en M' por inspección de casos:

$$\begin{aligned} g(\delta(s_0, 0)) &= g(s_0) = s_a = \delta'(g(s_0), 0) = \delta'(s_a, 0) \\ g(\delta(s_0, 1)) &= g(s_1) = s_b = \delta'(g(s_0), 1) = \delta'(s_a, 1) \\ g(\delta(s_1, 0)) &= g(s_0) = s_a = \delta'(g(s_1), 0) = \delta'(s_b, 0) \\ g(\delta(s_1, 1)) &= g(s_2) = s_b = \delta'(g(s_1), 1) = \delta'(s_b, 1) \\ g(\delta(s_2, 0)) &= g(s_0) = s_a = \delta'(g(s_2), 0) = \delta'(s_b, 0) \\ g(\delta(s_2, 1)) &= g(s_1) = s_b = \delta'(g(s_2), 1) = \delta'(s_b, 1) \\ f_0(s_0) &= 0 = f'_0(g(s_0)) = f'_0(s_a) = 0 \\ f_0(s_1) &= 1 = f'_0(g(s_1)) = f'_0(s_b) = 1 \\ f_0(s_2) &= 1 = f'_0(g(s_2)) = f'_0(s_c) = 1 \end{aligned}$$

□

Ejercicio: en esta sección se ha utilizado como modelo de autómatas a los autómatas finitos reconocedores. Definir las nociones de Homomorfismo e Isomorfismo para los autómatas traductores de Mealy y de Moore. *Sugerencia:* dada la convención notacional adoptada respecto de los estados de aceptación en AFR es conveniente comenzar por los AFT de Moore.

1.4.1 Autómata Finito Cociente

DEFINICIÓN 1.4.1 Sea g un homomorfismo de un autómata $M = (S, \Sigma, s_0, \delta, f_0)$ en un autómata $M' = (S', \Sigma, s'_0, \delta', f'_0)$, definimos la relación binaria ρ , sobre S , de la siguiente forma:

$$s_i \rho s_j \text{ si y solo si } g(s_i) = g(s_j)$$

■

EJEMPLO 1.4.2 Tomando los dos autómatas finitos del último ejemplo, el homomorfismo g determina en M dos clases: $[s_0]$ y $[s_1]$, $s_1 \rho s_2$ ya que $g(s_1) = g(s_2) = s_b$ entonces $[s_1] = \{s_1, s_2\}$. $\square$

Dos estados de M están relacionados por ρ cuando por el homomorfismo tienen asociado el mismo estado en M' ; ρ es una relación de congruencia sobre S , por lo tanto ρ particiona a S en clases de equivalencia. Trataremos las clases de estados como estados de una máquina que denominaremos máquina cociente M/g .

DEFINICIÓN 1.4.2 sean $M = (S, \Sigma, s_0, \delta, f_0)$ y M' dos AFR y g un homomorfismo entre los autómatas M y M' . Denominaremos *Autómata cociente* $M/g = (S'', \Sigma, [s_0], \delta'', f'')$ al AFR tal que:

S'' es el conjunto de estados de M/g , tal que cada uno es una clase de equivalencia $[s]$ de estados de M determinada por g . Todos los elementos de $[s]$ tienen asociado el mismo estado en M' y la misma salida.

Σ el alfabeto de entrada de M .

El estado inicial $[s_0]$ es la clase del estado inicial de M .

f'' es la función de salida, $f''([s_i]) = f_0(s_i)$.

δ'' es la función de próximo estado definida mediante $\delta''([s_j], i) = [\delta(s_j, i)]$. Es decir, δ'' mapea bajo g al mismo estado de M' , a todos los estados s_n con idéntica entrada i . $\blacksquare$

Analicemos la función de próximo estado δ'' de M/g . Consideremos un estado $[s_j]$ de M/g y una entrada i . En el autómata M el próximo estado de s_j bajo i esta dado por $\delta(s_j, i)$. Aquí la función de próximo estado mapea $[s_j]$ bajo i en la clase $[\delta(s_j, i)]$. Debemos verificar ahora que la función δ'' esté bien definida, es decir, que su resultado sea independiente del representante de la clase elegido. Si $s_n \in [s_j]$ entonces

$$s_n \rho s_j \text{ si y solo si } g(s_n) = g(s_j) \quad (1.5)$$

además, por la definición del homomorfismo g

$$g(\delta(s_j, i)) = \delta''(g(s_j), i) \quad (1.6)$$

y

$$g(\delta(s_n, i)) = \delta''(g(s_n), i) \quad (1.7)$$

luego

$$g(\delta(s_j, i)) \stackrel{1.6}{=} \delta''(g(s_j), i) \stackrel{1.5}{=} \delta''(g(s_n), i) \stackrel{1.7}{=} g(\delta(s_n, i))$$

Por lo tanto la función de próximo estado en M mapea bajo g al mismo estado de M' , a todos los estados $s_n \in [s_j]$ con idéntica entrada i . Luego estos estados pertenecen a la misma clase. De allí que podemos definir el próximo estado de una clase bajo la entrada i como la clase del próximo estado de un representante de ella.

EJEMPLO 1.4.3 Determinar en el autómata anterior los estados de M/g . El conjunto de los estados está dado por $S'' = \{[s_0], [s_1]\}$ donde: $[s_0] = \{s_0\}$ y $[s_1] = \{s_1, s_2\}$. La función de transición y la función de aceptación es definida de la siguiente manera:

δ''	0	1	f''_0
$[s_0]$	$[s_0]$	$[s_1]$	0
$[s_1]$	$[s_0]$	$[s_1]$	1

El diagrama de estados para M/g es el dado en la figura 1.5:

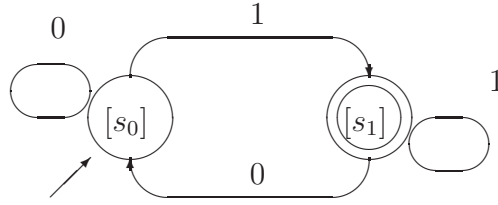


Figura 1.5: Ejemplo de autómata cociente

□

DEFINICIÓN 1.4.3 Se dice que g es un *isomorfismo de un AFR* $M'' = (S'', \Sigma, s''_0, \delta'', f''_0)$ en otro AFR $M' = (S', \Sigma, s'_0, \delta', f'_0)$ si g es una función $g : S'' \rightarrow S'$ tal que :

1. g es un homomorfismo entre autómatas,
2. g es una biyección

■

Si $g : M'' \rightarrow M'$ es un isomorfismo, el número de estados en S'' es igual al de S' y existe g^{-1} . Dos autómatas finitos isomorfos son estructuralmente idénticos.

TEOREMA 1.4.1 (Homomorfismos sobre Autómatas) Sea g un homomorfismo sobreyectivo de un AF M en un AF M' ; entonces la imagen homomorfa de M , M' , es isomorfa al AF cociente M/g .

■

Este teorema indica que dado un AF M , el AF cociente M/g obtenido como se mostró anteriormente es isomorfo o esencialmente idéntico a M' , la imagen homomorfa de M bajo g . Una de las aplicaciones de este teorema es en la obtención de un AF minimal. En tal caso debemos encontrar el homomorfismo g y hallar el AF cociente M/g .

Demostración 4 Sea g un homomorfismo sobreyectivo de M a M' ; para probar que la imagen homomorfa, M' , es isomorfa a M/g definimos un isomorfismo h de M/g a M' . Sea $[s]$ un estado de M/g . Definimos h , como $h([s]) = g(s)$ donde cualquier representante de $[s]$ tendrá la misma imagen bajo g . Queremos demostrar que:

1. h es una biyección. Para probar que h es biyectiva, debemos probar que h es sobreyectiva e inyectiva:

Para probar que h es sobreyectiva, consideremos un estado $s' \in S'$, $s' = g(s)$ para algún $s \in S$. $[s]$ es un estado de M/g y por definición de h , $h([s]) = g(s) = s'$. Por lo tanto es sobreyectiva (como h es definido en función de g , h será sobreyectivo si g lo es).

Para probar que h es inyectiva, supongamos que $h([s_m]) = h([s_n])$. Por definición de h , si $h([s_m]) = h([s_n])$ entonces $g(s_m) = g(s_n)$ y por definición de g , s_m y s_n están en la misma clase y $[s_m] = [s_n]$. Por lo tanto h es inyectiva.

2. h es un homomorfismo. Es decir deseamos demostrar que

$$h([s_0]) = s'_0, h(\delta''([s], i)) = \delta'(h([s]), i) \text{ y } f''_0([s]) = f'_0(h([s]))$$

Primero debe observarse que por definición de h tendremos $h([s_0]) = g(s_0) = s'_0$. Además,

$$h(\delta''([s], i)) \stackrel{(1)}{=} h([\delta(s, i)]) \stackrel{(2)}{=} g(\delta(s, i)) \stackrel{(3)}{=} \delta'(g(s), i) \stackrel{(2)}{=} \delta'(h([s]), i)$$

(1) por definición de δ .

(2) por definición de h .

(3) por ser g homomorfismo de M en M' .

$$f''_0([s]) \stackrel{(1)}{=} f_0(s) \stackrel{(2)}{=} f'_0(g(s)) \stackrel{(3)}{=} f'_0(h([s]))$$

(1) por definición de ρ .

(2) por ser g homomorfismo de M en M' .

(3) por definición del isomorfismo h .

Entonces estados correspondientes a una misma clase, bajo h llevan a estados correspondientes en M' . Como la función de salida de un estado $[s]$ de M/g se define como la salida de $g(s)$ en M' , entonces un estado en M/g y su correspondiente bajo h en M' tienen la misma salida. $\square$

EJEMPLO 1.4.4 Luego, en el ejemplo anterior, el isomorfismo h de M/g a M' es

$$h =_{def} \begin{cases} h([s_0]) = s_a \\ h([s_1]) = s_b \end{cases} \quad \square$$

EJEMPLO 1.4.5 Dados los autómatas finitos M, M' , definir, si es posible, el homomorfismo g , el AF cociente M/g y el isomorfismo de M/g a M' .

$$\begin{array}{ll} M = (S, \Sigma, \delta, s_0, f_0) & M' = (S', \Sigma, \delta', s'_0, f'_0) \\ S = \{s_0, s_1, s_2, s_3, s_4\} & S' = \{s_a, s_b, s_c\} \\ \Sigma = \{0, 1\} & \Sigma = \{0, 1\} \end{array}$$

δ	0	1	f_0
s_0	s_2	s_1	0
s_1	s_2	s_1	0
s_2	s_0	s_4	0
s_3	s_3	s_2	1
s_4	s_3	s_2	1

δ'	0	1	f'_0
s_a	s_b	s_a	0
s_b	s_a	s_c	0
s_c	s_c	s_b	1

Dados M y M' planteamos el siguiente homomorfismo $g : S \rightarrow S'$

$$g =_{def} \begin{cases} g(s_0) = s_a \\ g(s_1) = s_a \\ g(s_2) = s_b \\ g(s_3) = s_c \\ g(s_4) = s_c \end{cases}$$

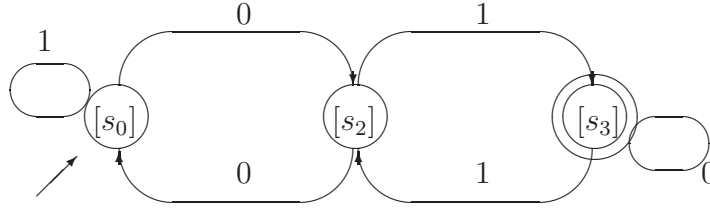
Los estados de M/g son: $[s_0] = \{s_0, s_1\}$, $[s_2] = \{s_2\}$ y $[s_3] = \{s_3, s_4\}$. Y el autómata finito M/g puede ser definido mediante $M/g = (S'', \Sigma, \delta'', [s_0], f''_0)$ donde:

$$\Sigma = \{0, 1\} \quad S'' = \{[s_0], [s_2], [s_3]\}$$

δ''	0	1	f_0''
$[s_0]$	$[s_2]$	$[s_0]$	0
$[s_2]$	$[s_0]$	$[s_3]$	0
$[s_3]$	$[s_3]$	$[s_2]$	1

El isomorfismo $h : S'' \rightarrow S'$ puede ser definido mediante

$$h =_{def} \begin{cases} h([s_0]) = s_a \\ h([s_2]) = s_b \\ h([s_3]) = s_c \end{cases}$$



□

Ejercicio: adaptar las definiciones de esta sección para los autómatas traductores de Mealy y de Moore.

1.4.2 Minimización de Autómatas Finitos

Podemos resumir el principal resultado de la sección anterior diciendo que toda imagen homomorfa de un AF M es esencialmente un AF cociente de M y todo Autómata cociente de M es la imagen homomorfa de M . Siempre que el conjunto de estados S de un AF pueda ser particionado y que los bloques de esa partición se usen para la definición de un nuevo AF, éste es llamado *autómata finito cociente*.

Hemos estudiado los AF y los homomorfismos sobre AF. Utilizaremos el concepto de autómata finito cociente para dar un método de minimización de los mismos. La noción intuitiva de minimizar es hallar un AF que tenga el mismo comportamiento con menor cantidad de estados. ¿Cuáles son las condiciones para la existencia del autómata finito cociente?

Todos los estados en un bloque dado de la partición deben tener, dado un símbolo $i \in \Sigma$, próximos estados bajo δ , que pertenezcan a su vez a un mismo bloque de la partición.

Todos los estados de un bloque dado de la partición deben tener la misma salida en el caso de AF Traductores. En el caso de los AF reconocedores todos los estados de un bloque deben o bien pertenecer o no pertenecer al conjunto F .

Naturalmente, una partición sobre un AF que cumpla con las condiciones anteriores determina el homomorfismo entre el AF dado y el AF cociente.

DEFINICIÓN 1.4.4 Dado un autómata finito $M = (S, \Sigma, s_0, \delta, f_0)$ y un estado $s_i \in S$ tal que $s_i \neq s_0$, se dice que s_i es un *estado inalcanzable* de M si no existe una cadena $w \in \Sigma^*$ tal que $\delta(s_0, w) = s_i$. ■

EJEMPLO 1.4.6 Determinar, si es posible, los estados inalcanzables de

$$M = (S, \Sigma, \delta, s_0, f_0)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$\Sigma = \{0, 1\}$$

δ	0	1	f_0
s_0	s_1	s_3	0
s_1	s_3	s_0	0
s_2	s_1	s_3	1
s_3	s_0	s_1	1

s_2 es estado inalcanzable. Para facilitar la detección de estados inalcanzables es útil realizar el grafo asociado al autómata. $\square$

DEFINICIÓN 1.4.5 Dos estados s_i, s_j de un AF M son *equivalentes* si para toda cadena de entrada, $w \in \Sigma^*$, $\delta(s_i, w) = s_k$ y $\delta(s_j, w) = s_l$, donde $f_0(s_k) = f_0(s_l)$. $\blacksquare$

Informalmente, dos estados s_i, s_j de un autómata finito M son equivalentes si para cualquier cadena $w \in \Sigma^*$, el efecto de M comenzando en el estado s_i con la cadena w es la misma, que comenzando en el estado s_j con la cadena w . Es decir, $f_0(\delta(s_i, w)) = f_0(\delta(s_j, w))$.

Ahora podemos afirmar que el problema de minimizar un autómata finito M se reduce al de encontrar los estados equivalentes, y con los bloques que éstos determinan definir un autómata finito cociente sobre M . El inconveniente es ¿Cómo probar con todas las cadenas pertenecientes a Σ^* dada la definición anterior de estados equivalentes?. Para solucionarlo debemos extender la noción de equivalencia entre estados.

DEFINICIÓN 1.4.6 Dos estados s_i, s_j de un AFR M son *k-equivalentes* si para $w \in \Sigma^*$ con $\text{long}(w) \leq k$, $\delta(s_i, w) = s_n$ y $\delta(s_j, w) = s_l$, donde $f_0(s_n) = f_0(s_l)$. $\blacksquare$

DEFINICIÓN 1.4.7 Sean R y R' dos relaciones de equivalencia definidas sobre un conjunto S diremos que R *refina* a R' , si siempre que xRy entonces $xR'y$ para todo $x, y \in S$. $\blacksquare$

Observemos que se dice que R refina R' si las clases de equivalencia de R están formadas por particionar las clases de R' en clases más pequeñas. En nuestro contexto la k -equivalencia refina la $(k-1)$ -equivalencia. Mas aún, en este contexto una k -equivalencia refina a toda r -equivalencia tal que $r < k$.

Algoritmo (para minimizar un AF):

Entrada: un AF $M = (S, \Sigma, \delta, s_0, F)$ determinista completo

Salida: el AF M minimizado

1. Eliminar del conjunto de estados S a los estados inalcanzables y llamar a este nuevo conjunto T .
2. Particionar T en dos clases formadas por los estados 0-equivalentes. Por tratarse de un AFR tendremos una clase de estados aceptadores y otra de estados no aceptadores.

3. Asumir $k = 0$.

4. Repetir:

- Determinar las clases $(k+1)$ -equivalentes como un refinamiento de las k equivalentes, es decir: s_i, s_j son $(k+1)$ -equivalentes si y solo si s_i, s_j son k -equivalentes y $\delta(s_i, i), \delta(s_j, i)$ son 1-equivalentes, para todo $i \in \Sigma$.
- Incrementar k en 1.

hasta que las clases $(k+1)$ -equivalentes sean iguales a las k -equivalentes.

5. Usar las clases k -equivalentes determinadas para definir el autómata cociente de M . $\square$

Observación: éste procedimiento puede extenderse a autómatas finitos traductores.

EJEMPLO 1.4.7 Usar el algoritmo dado para minimizar el siguiente autómata M .

$$M = (S, \Sigma, \delta, s_0, f_0)$$

$$S = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6\}$$

$$\Sigma = \{0, 1\}$$

δ	0	1	f_0
s_0	s_2	s_3	0
s_1	s_3	s_2	1
s_2	s_0	s_4	0
s_3	s_1	s_5	1
s_4	s_6	s_5	1
s_5	s_2	s_0	0
s_6	s_4	s_0	1

Utilicemos el procedimiento dado para minimización de autómatas finitos reconocedores (Paso 1):

$$T = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6\}$$

En M no existen estados inalcanzables, entonces $T = S$. Los estados 0-equivalentes de M , notados mediante $|0|$, son (Paso 2):

$$|0| = \{s_0, s_2, s_5\}\{s_1, s_3, s_4, s_6\}$$

Por aplicación repetida del proceso de refinamiento (Paso 4) obtenemos:

$$\begin{aligned} |1| &= \{s_0, s_2\}\{s_5\}\{s_1, s_3, s_4, s_6\} \\ |2| &= \{s_0, s_2\}\{s_5\}\{s_1, s_6\}\{s_3, s_4\} \\ |3| &= \{s_0, s_2\}\{s_5\}\{s_1, s_6\}\{s_3, s_4\} \end{aligned}$$

Como las clases 2-equivalentes son iguales a las 3-equivalentes se termina el refinamiento de clases. Renombramos los estados para construir el autómata minimizado (Paso 5), es decir el autómata cociente M/g donde g es el homomorfismo de los estados de M a las clases dadas. Los estados de M/g serán:

$$\begin{aligned} [s_0] &= \{s_0, s_2\} \\ [s_1] &= \{s_1, s_6\} \\ [s_3] &= \{s_3, s_4\} \\ [s_5] &= \{s_5\} \end{aligned}$$

$$M/g = (S'', \Sigma, \delta'', [S_o], f''_0)$$

$$S' = \{[s_0], [s_1], [s_3], [s_5]\}$$

$$\Sigma = \{0, 1\}$$

δ'	0	1	f''_0
$[s_0]$	$[s_0]$	$[s_3]$	0
$[s_1]$	$[s_3]$	$[s_0]$	1
$[s_3]$	$[s_1]$	$[s_5]$	1
$[s_5]$	$[s_0]$	$[s_0]$	0

□

Como conclusión obtenemos que el método de minimización de AF encuentra exactamente un AF cociente M/g donde el homomorfismo g es el que particiona a los estados de M en estados k -equivalentes. Este procedimiento de minimización es debido a Moore.

Ejercicio: investigar que ocurre si todos los estados del autómata a minimizar son aceptadores.

Ejercicio: adaptar los resultados de esta sección para AF Traductores.

1.5 Expresiones Regulares

Un tema central en teoría de computación es la representación finita de lenguajes. Naturalmente, todo lenguaje finito puede tener como representación finita la enumeración, pero ¿Que sucede cuando el lenguaje es infinito? En el caso que el lenguaje considerado es el de los identificadores válidos en Pascal si no tenemos en cuenta restricciones de longitud. El problema que se plantea es cómo obtener una notación compacta que describa los lenguajes, en particular los lenguajes reconocidos por autómatas finitos, tal que nos permita representar aún lenguajes infinitos. Hasta ahora hemos explorado la pregunta inversa, esto es, dada la descripción informal de un lenguaje como construir un AF que lo reconozca. Nuestra misión ahora será encontrar una forma de expresar lenguajes tal que exista un procedimiento para obtener la representación de todas las cadenas de un lenguaje a partir de uno de sus autómatas reconocedores. Tal forma de expresar los lenguajes regulares es la de las denominadas “expresiones regulares”.

DEFINICIÓN 1.5.1 Las *expresiones regulares (ER)* sobre un alfabeto Σ son cadenas sobre el alfabeto $A = \Sigma \cup \{(\cdot), \bullet, +, \star, \lambda, \emptyset\}$ definidas recursivamente como sigue:

1. λ es una ER.
2. $\emptyset$ es una ER.
3. Todo símbolo $i, i \in \Sigma$, es una ER.
4. Si α y β son ER, entonces $(\alpha \bullet \beta)$ también es una ER.
5. Si α y β son ER, entonces $(\alpha + \beta)$ también es una ER.
6. Si α es una ER, entonces α^* también es una ER. □

Notación: De ser α una expresión con mas de un símbolo lo notaremos $(\alpha)^*$.

EJEMPLO 1.5.1 Las siguientes son expresiones regulares sobre el alfabeto $\Sigma = \{a, b\}$: $\lambda, a, a \bullet b, (a \bullet b), (a \bullet b + b \bullet b)^*, (a^* \bullet b \bullet (a \bullet b)^*)$. □

DEFINICIÓN 1.5.2 Denominaremos *Lenguajes Regulares* a la clase de los lenguajes representables mediante expresiones regulares. ■

La relación entre expresiones regulares y los lenguajes que representan queda establecida por una función $\mathcal{L}$ tal que si α es una ER entonces $\mathcal{L}(\alpha)$ es el lenguaje representado por α . $\mathcal{L}$ es una función cuyo dominio es el conjunto de las ER posibles sobre un alfabeto Σ y su rango el conjunto de lenguajes definidos sobre Σ .

DEFINICIÓN 1.5.3 Construimos la función $\mathcal{L} : A \rightarrow \Sigma^*$ como sigue :

1. $\mathcal{L}(\lambda) = \{\lambda\}$
2. $\mathcal{L}(\emptyset) = \emptyset$
3. Para cada símbolo $a \in \Sigma$ $\mathcal{L}(a) = \{a\}$
4. Si α y β son ER, $\mathcal{L}(\alpha \bullet \beta) = \mathcal{L}(\alpha) \bullet \mathcal{L}(\beta)$.
5. Si α y β son ER, $\mathcal{L}(\alpha + \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$.
6. Si α es una ER, entonces $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$. ■

Nota: Aquí adoptamos la notación que es de uso mas extendido en los textos de computación para facilitar el uso de los mismos por parte del lector. No obstante debe observarse que debería notarse con distintos símbolos las operaciones sobre cadenas que las operaciones sobre lenguajes. El símbolo de concatenación ($\bullet$) será omitido cuando ello no provoque ambigüedad. Luego a la última cadena la reescribiremos como $(a^*b(ab)^*)$. Además, usaremos el signo ‘+’ como supraíndice para indicar repetición una o mas veces. En consecuencia a^+ será una forma de abreviar aa^* .

Observación: Debemos distinguir la expresión $\mathcal{L}(\emptyset)$, lenguaje sin elementos, de la expresión $\mathcal{L}(\lambda)$, lenguaje formado por un único elemento: λ .

Cabe destacar que un lenguaje regular puede ser descripto por más de una ER.

EJEMPLO 1.5.2

$$(0 + 1)^* = ((0 + 1^*)^* + (01)^*)^*$$

□

Hasta ahora no existe un algoritmo eficiente para decidir si dos ER son equivalentes. Naturalmente esto no prohíbe su existencia.

EJEMPLO 1.5.3 Cómo son las cadenas pertenecientes al lenguaje $\mathcal{L}((a + b)^*a)$?

$$\mathcal{L}(((a + b)^*a)) = \mathcal{L}((a + b)^*)\mathcal{L}(a) = \mathcal{L}(a + b)^*\{a\} = (\mathcal{L}(a) \cup \mathcal{L}(b))^*\{a\} = \{a, b\}^*\{a\}$$

Es decir son todas las cadenas $w \in \{a, b\}^*$ que terminan con a .

□

Para determinar si, dada una ER α , una cadena w pertenece a $\mathcal{L}(\alpha)$, debemos realizar un proceso constructivo a partir de la ER tendiente a obtener la cadena w .

EJEMPLO 1.5.4 $10100010 \in \mathcal{L}((0^*10)^*)$ pues se puede obtener como 10 (utilizando 0^010), 1010 (utilizando 0^010), concluyendo con 10100010 (mediante 0^210). □

Ejercicio: verificar que $011100 \notin \mathcal{L}((0 + (11)^*)^*)$ y $000111100 \in \mathcal{L}(((011 + 11)^*(00)^*)^*)$

Ejercicio: determinar que lenguajes son representados por las siguientes ER definidas sobre el alfabeto $\{0, \dots, 9, .\}$. Para mayor claridad $d = \{0, \dots, 9\}$, es decir, el conjunto de los dígitos arábigos.

$$d.d^*$$

$$(d^*.dd^* \cup d^*d.d^*)$$

Nota: en este ejercicio se usa “.” para denotar el punto decimal, no debe confundirse con el de concatenación, el cual se usa en forma implícita.

1.5.1 Teorema de Kleene

Hemos introducido los lenguajes regulares pues son exactamente los lenguajes que reconoce un AF. Demostraremos ahora que cualquier lenguaje reconocido por un AF es regular y cualquier lenguaje regular puede ser reconocido por un AF. Este resultado es debido a Kleene y data de 1956.

TEOREMA 1.5.1 Un lenguaje L es regular si y sólo si es reconocido por un AF. ■

Demostración 5 ($\Rightarrow$) La clase de lenguajes regulares es la clase más chica de lenguajes que contiene al conjunto $\emptyset$, a λ , a los símbolos del alfabeto y es cerrada bajo unión, concatenación y estrella de Kleene. La prueba será por construcción del AF.

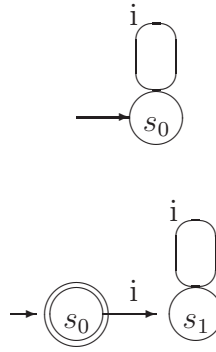
1. El menor AF para $L = \emptyset$ es

$$M = (S, \Sigma \cup \lambda, \delta, s_0, F)$$

$$S = \{s_0\}$$

$$i \in \Sigma$$

$$F = \emptyset$$



2. El menor autómata finito que reconoce λ es

$$M = (S, \Sigma \cup \lambda, \delta, s_0, F)$$

$$S = \{s_0, s_1\}$$

$$F = \{s_0\}$$

$$i \in \Sigma$$

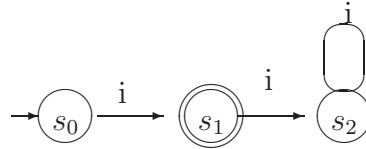
3. El menor autómata finito que reconoce a todo el alfabeto Σ es

$$M = (S, \Sigma \cup \lambda, \delta, s_0, F)$$

$$S = \{s_0, s_1, s_2\}$$

$$F = \{s_1\}$$

$$i \in \Sigma$$



4. Veamos que la unión de expresiones regulares también es reconocible a través de un autómata finito. Sean L_1 y L_2 dos lenguajes regulares aceptados por autómatas finitos M_1 y M_2 respectivamente, posiblemente no deterministas. Queremos probar que $L_1 \cup L_2$ es un lenguaje aceptado por un autómata finito. Lo demostraremos por construcción:

Sean $M_1 = (S_1, \Sigma \cup \lambda, \delta_1, s_1, F_1)$ y $M_2 = (S_2, \Sigma \cup \lambda, \delta_2, s_2, F_2)$, sin pérdida de generalidad, podemos asumir que los conjuntos de estados S_1 y S_2 son disjuntos (si no lo fueran, renombramos los estados de alguno de ellos). Construimos un autómata finito M no determinista que acepte $L(M_1) \cup L(M_2)$. $M = (S, \Sigma \cup \{\lambda\}, \delta, s, F)$, donde:

$$S = S_1 \cup S_2 \cup \{s\},$$

s es el nuevo estado inicial, $s \notin S_1 \cup S_2$, en particular es distinto de s_1 y s_2 ,

$$F = F_1 \cup F_2$$

$$\delta = \delta_1 \cup \delta_2 \cup \{ \delta(s, \lambda) = \{s_1, s_2\} \}$$

M comienza en el estado s , eligiendo el estado inicial de M_1 o el estado inicial de M_2 . Esto queda dado por la transición $\delta(s, \lambda) = \{s_1, s_2\}$, donde λ debe interpretarse como “no se consume cadena de entrada”. Luego se comporta como M_1 o M_2 , dependiendo de la elección, i.e, el AF M es no determinista. Formalmente, si $w \in \Sigma^*$, entonces:

$$\delta(s, w) \supseteq \{q\}, q \in F, \text{ si y solo si: } \delta_1(s_1, w) \supseteq \{q\}, q \in F_1 \text{ o } \delta_2(s_2, w) \supseteq \{q\}, q \in F_2$$

es decir, una cadena es aceptada, si es aceptada por cualquiera de las dos máquinas M_1 o M_2 . Por lo tanto $L(M) = L(M_1) \cup L(M_2)$. CQD.

5. Procederemos análogamente a demostrar que la concatenación de expresiones regulares tambien es reconocible por un autómata finito. Sean M_1 y M_2 dos autómatas finitos, construiremos un AF M tal que $L(M) = L(M_1) \bullet L(M_2)$. Intuitivamente, M opera simulando M_1 y luego pasa a M_2 y lo simula. Formalmente, dados $M_1 = (S_1, \Sigma \cup \lambda, \delta_1, s_1, F_1)$ y $M_2 = (S_2, \Sigma \cup \lambda, \delta_2, s_2, F_2)$, donde S_1 y S_2 son conjuntos disjuntos. Construimos $M = (S, \Sigma, \delta, s, F)$ donde:

$$S = S_1 \cup S_2$$

$$s = s_1$$

$$F = F_2$$

$$\delta = \delta_1 \cup \delta_2 \cup \{\delta(s_i, \lambda) = s_2, \text{ para todo } s_i \in F_1\}$$

Sea $w \in \Sigma^*$, entonces $\delta(s, w) = q, q \in F$ si y solo si $w = w_1 w_2$, $w_1, w_2 \in \Sigma^*$ y w_1, w_2 son tales que $\delta_1(s_1, w_1) = p, p \in F_1$; $\delta(p, \lambda) = s_2$ y $\delta_2(s_2, w_2) = q, q \in F_2$. Por lo tanto, $L(M) = L(M_1)L(M_2)$. CQD.

6. Finalmente solo resta ver que la Estrella de Kleene de una expresión regular es reconocible a traves de un autómata finito. Sea M_1 un AF, construiremos un AFND M tal que $L(M) = L(M_1)^*$. M es tal que se comporta como M_1 hasta que una porción de la cadena de entrada alcanza un estado final. En ese punto M puede elegir volver a empezar. Formalmente, dado $M_1 = (S_1, \Sigma \cup \lambda, \delta_1, s_1, F_1)$ construimos M tal que tiene todos los estados de M_1 y un nuevo estado inicial s'_1 . Este nuevo estado también es aceptador, para poder reconocer λ .

$$S = S_1 \cup \{s'_1\}$$

$$s'_1 \text{ es el estado inicial, } s'_1 \notin S_1$$

$$F = F_1 \cup \{s'_1\}$$

$$\delta = \delta_1 \cup \{\delta(s'_1, \lambda) = s_1\} \cup \{\delta(s_i, \lambda) = s_1 \text{ para cada } s_i \in F_1\}$$

Es válido preguntarse acerca de la necesidad de agregar s'_1 . Una pregunta que puede surgir es ¿No se obtiene el mismo resultado transformando s_1 en aceptador y agregando arcos de los estados miembros de F_1 a s_1 ? El problema de proceder de este modo es que cambiaríamos $L(M_1)$ ya que el estado s_1 podría ser un nodo sobre el cual hay arcos incidentes, y entonces M_1 pasaría a aceptar otras cadenas ademas de las que inicialmente aceptaba. Es decir, estaríamos cambiando la definición del autómata de partida.

Probemos por inspección de casos de M que si $w \in L(M)$ luego $w \in L(M_1)^*$:

si $w = \lambda$, $w \in L(M_1)^*$ pues s'_1 , estado inicial, pertenece a F ;

si $w = w_1 \dots w_k$, $w_i \in \Sigma$, $1 \leq i \leq k$. Sabemos que $\delta_1(s_1, w) = f_i, f_i \in F$, M utilizará $\delta(s'_1, w) = \delta(s_1, w)$, luego $w \in L(M_1)^*$.

Demostremos que si $w \in L(M_1)^*$ entonces $w \in L(M)$:

si $w = \lambda$, entonces $w \in L(M)$ pues s'_1 es un estado final.

si $w = w_1 \dots w_k$ para $w_i \in L(M_1)$ entonces $w \in L(M)$, pues para $f_1, f_2, \dots, f_k \in F_1$

$$\begin{aligned} \delta(s'_1, w_1 \dots w_k) &= \delta(s_1, w_1 \dots w_k) = \delta(f_1, w_2 \dots w_k) \\ &= \delta(s_1, w_2 \dots w_k) = \delta(f_2, w_3 \dots w_k) \\ &= \delta(s_1, w_3 \dots w_k) = \dots \\ &= f_k \end{aligned}$$

Luego, $L(M) = L(M_1)^*$. CQD. ($\Rightarrow$)

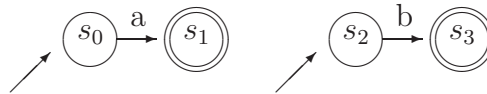
La otra mitad de la demostración ($\Leftarrow$), clásicamente es desarrollada utilizando inducción, no será dada aquí remitiendo al lector a otras fuentes ([HU79] y [Coh91]). Dejaremos incompleta esta demostración por el momento pero compensaremos su falta en la próxima sección. Luego de considerar algunos ejemplos para ilustrar el uso de la mitad de la demostración dada consideraremos la otra mitad mediante un método constructivo que permite determinar la expresión regular asociada a un autómata finito. $\square$

Consideraremos ahora algunos ejemplos que esclarecen la utilidad del caracter constructivo de la primer mitad de la demostración recién dada. Respecto del uso de la primera mitad de la demostración ofrecemos el siguiente ejemplo, en el cual la misma es utilizada para guiar la construcción de un AF que reconoce una ER dada.

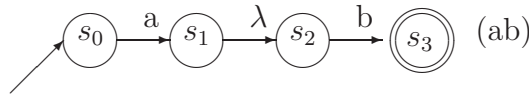
EJEMPLO 1.5.5 Sea la ER $(ab + aab)^*$, se desea construir un AF que acepte el lenguaje L representado por dicha expresión.

Nota: con el objeto de simplificar la explicación no se darán AF totalmente especificados. Además, también se omite la definición formal de los mismos. Su definición detallada queda como ejercicio para el lector.

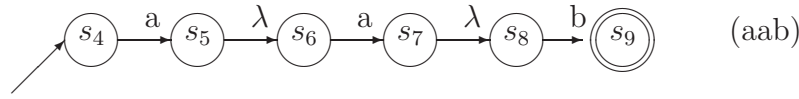
1. Paso: a, b



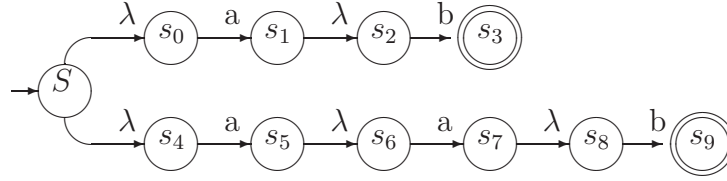
2. Paso: concatenación para obtener ab y aab



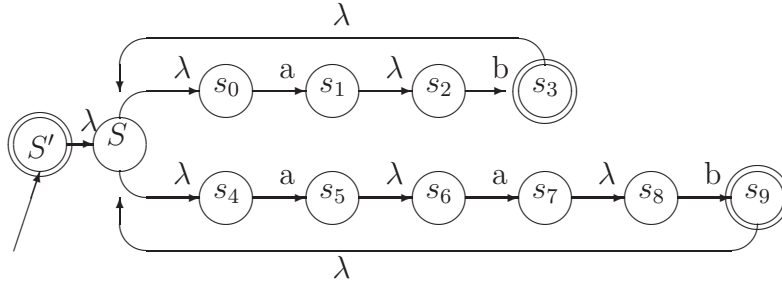
debe observarse que en este último grafo se utilizó renombramiento de estados a partir de los grafos anteriores



3. Paso: unión para obtener $(ab + aab)$



4. Paso: Estrella de Kleene $(ab + aab)^*$



Debe notarse que el agregado del estado inicial extra es necesario para evitar afectar la función de transición del autómata finito a partir del que se construyó, ya que este pasa a ser aceptador para aceptar λ . $\square$

1.5.2 Determinación de la Expresión Regular

Al plantear la equivalencia entre los lenguajes reconocidos por AF y los lenguajes representados por ER quedó sin demostración el hecho de que dado un autómata finito existe una Expresión Regular que representa el mismo lenguaje. Esto no quiere decir que este aspecto de la relación carezca de interés en este contexto. Veamos los siguientes ejemplos en los cuales se requiere obtener la ER asociada a un dado AF.

EJEMPLO 1.5.6 escribir las expresiones regulares para los lenguajes reconocidos por los autómatas finitos dados a continuación.

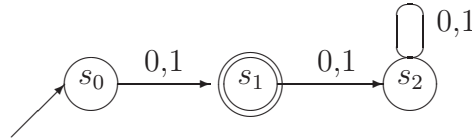
Sea $M = (S, \Sigma, \delta, s_0, F)$ donde:

$$S = \{s_0, s_1, s_2\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{s_1\}$$

δ	0	1
s_0	s_1	s_1
s_1	s_2	s_2
s_2	s_2	s_2



Analicemos su comportamiento, el autómata tiene un solo estado aceptador y hay solo una forma de acceder a él: a través del arco etiquetado con los símbolos 0 y 1. Si luego de procesar el símbolo 0 o el símbolo 1 se recibe otro, M pasa a un estado no aceptador. Luego, es claro que la expresión regular reconocida por el autómata precedente es $(0 + 1)$.

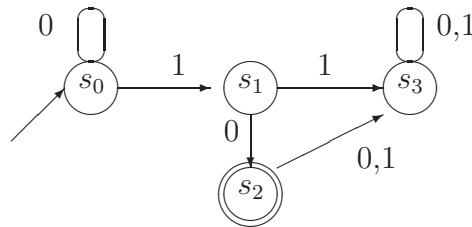
Sea $M = (S, \Sigma, \delta, s_0, F)$ donde:

$$S = \{s_0, s_1, s_2, s_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{s_2\}$$

δ	0	1
s_0	s_0	s_1
s_1	s_2	s_3
s_2	s_3	s_3
s_3	s_3	s_3



En este caso la determinación de la expresión regular reconocida por el AF requiere un poco más de esfuerzo que en el caso anterior. De todas formas puede advertirse sin mucha dificultad que existe un camino entre el estado inicial y el estado aceptador, para recorrerlo se debe procesar primero el símbolo 1 y luego el 0. Previamente a procesar la cadena 10 puede haberse procesado, a través del estado inicial una secuencia posiblemente vacía de ceros. Podemos resumir esto concluyendo que la expresión regular reconocida es 0^*10 . $\square$

A partir de estos ejemplos sencillos, puede pensarse que la determinación de la expresión regular no requiere mucho esfuerzo y puede realizarse sin un método como guía. Pero a medida que los AF crecen en complejidad, también crece la incertidumbre acerca de la ER que reconocen.

A continuación presentaremos un método sistemático para hallar la ER asociada a un AF. Antes de dar el algoritmo que hará las veces de demostración es necesario la enunciación del siguiente lema:

LEMA 1.5.1 Si ω , β y γ son expresiones regulares sobre un alfabeto Σ , $\gamma \neq \lambda$, entonces la ecuación $\omega = \beta + \omega\gamma$ tiene una única solución y ésta es $\omega = \beta\gamma^*$ $\blacksquare$

Demostración 6 Queda propuesta como ejercicio. Ayuda: reemplazar la hipótesis en uno de los términos de la ecuación y transformarla a través de equivalencias hasta obtener el otro.

Ya que existe una forma de resolver ecuaciones sobre cadenas, debemos encontrar una forma de notar la información del autómata finito en ecuaciones y luego aplicar la resolución al sistema. Las variables del sistema representan el conjunto de cadenas que llegan a cada estado, es decir para cada estado s_j tendremos una variable que llamaremos ω_{s_j} . También haremos uso, sin demostración previa, de algunas reglas sencillas de conversión de ER tales como $(a + b)c = ac + bc$ o $a\lambda = a$.

Algoritmo (para determinar la expresión regular a partir de un autómata finito):

Entrada: un AF, $M = (S, \Sigma, s_0, \delta, F)$

Salida: una ER que denota el Lenguaje Regular reconocido por M

- Se plantea una ecuación por cada estado, como unión de n términos. Cada término representa un tipo de cadena que llega al estado; por lo tanto, existe un término por cada arco incidente. Estos términos son la concatenación de la variable correspondiente al estado origen del arco con el símbolo que rotula al arco. Para $\delta(s_j, a) = s_i$, uno de los términos de la ecuación ω_{s_i} será: $\omega_{s_j} \bullet a$. Es decir, $\omega_{s_i} = \omega_{s_j} \bullet a + \dots$

En la ecuación planteada para el estado inicial se agrega el término λ .

- Se despejan las ecuaciones según el lema: Si $\omega = \beta + \omega\gamma$ entonces $\omega = \beta\gamma^*$
- La ER es la unión de las soluciones para todos los estados aceptadores del AF.

□

Antes de ilustrar su uso consideremos algunos aspectos del uso y alcance de este algoritmo. En primer término es necesario mencionar que la solución al sistema de ecuaciones existe y es única, hecho que no demostraremos aquí. Una fuente de consulta acerca de estos resultados es [DDQ78].

Puede observarse que el enunciado contiene la hipótesis $\gamma \neq \lambda$. Esto no permite su uso en AF que tenga transiciones etiquetadas con λ . No obstante, el lector advertirá que esto no restringe su aplicabilidad de algún modo importante ya que para todo AF con λ transiciones puede obtenerse otro equivalente sin ellas.

El algoritmo permite su uso con AF no deterministas. De todos modos si se desea puede obtenerse primero el AF determinista y luego aplicar el algoritmo de resolución de ecuaciones. No es estrictamente necesario utilizar AF minimizados pero es deseable ya que esto conducirá a la obtención de una ER más sencilla. También hay que tener en cuenta que la simplificación de las ecuaciones del sistema puede realizarse en muchos ordenes posibles. El orden en que se consideren también tendrá efecto en la simplicidad de uso del procedimiento y en la simplicidad de la expresión regular obtenida.

Ejercicio: ¿Es estrictamente necesario plantear una ecuación para todo estado del autómata?

EJEMPLO 1.5.7 sea $\Sigma = \{a, b\}$, desarrollar un AF que reconozca el lenguaje

$$L = \{w \in \Sigma^* \mid w \text{ tiene } 3k + 1 \text{ b's, } k \geq 0\}$$

Para tal fin definimos el autómata:

$$\begin{aligned}
 M &= (S, \Sigma, \delta, s_0, F) \\
 S &= \{s_0, s_1, s_2\} \\
 \Sigma &= \{a, b\} \\
 F &= \{s_1\} \\
 \delta &= \begin{array}{c|cc} & a & b \\ \hline s_0 & s_0 & s_1 \\ s_1 & s_1 & s_2 \\ s_2 & s_2 & s_0 \end{array}
 \end{aligned}$$

Para determinar la ER que representa el lenguaje L utilizaremos el algoritmo dado. Siguiendo el algoritmo tendremos 3 ecuaciones con variables: $\omega_{s_0}, \omega_{s_1}, \omega_{s_2}$. Comenzando por s_0 , los arcos incidentes son los que provienen de s_0 y s_2 ya que $\delta(s_0, a) = s_0$ y $\delta(s_2, b) = s_0$, entonces:

$$\begin{aligned}
 \omega_{s_0} &= \omega_{s_0}a + \omega_{s_2}b + \lambda \\
 \omega_{s_1} &= \omega_{s_0}b + \omega_{s_1}a \\
 \omega_{s_2} &= \omega_{s_2}a + \omega_{s_1}b
 \end{aligned}$$

En este caso aplicamos el lema a las tres ecuaciones:

$$\omega_{s_0} = (\lambda + \omega_{s_2}b)a^* \quad (1.8)$$

pues $\gamma = a$ y $\beta = \omega_{s_2}b + \lambda$

$$\omega_{s_1} = (\omega_{s_0}b)a^* \quad (1.9)$$

pues $\gamma = a$ y $\beta = \omega_{s_0}b$

$$\omega_{s_2} = (\omega_{s_1}b)a^* \quad (1.10)$$

pues $\gamma = a$ y $\beta = \omega_{s_1}b$

Reemplazando 1.8 en 1.9, se obtiene $\omega_{s_1} = (\lambda + \omega_{s_2}b)a^*ba^*$, es decir, $\omega_{s_1} = a^*ba^* + \omega_{s_2}ba^*ba^*$. Reemplazando 1.10 en la igualdad anterior obtenemos $\omega_{s_1} = a^*ba^* + \omega_{s_1}ba^*ba^*ba^*$ lo cual nos permite deducir por medio del lema que $\omega_{s_1} = a^*ba^*(ba^*ba^*ba^*)^*$. Entonces la ER que representa el lenguaje L es $a^*ba^*(ba^*ba^*ba^*)^*$. $\square$

Ejercicio: tomando la idea del autómata finito anterior desarrollar uno que reconozca el lenguaje $L = \{w \in \Sigma^* | w \text{ tiene } 3k + 2 \text{ b's, } k \geq 0\}$. Utilizar el método dado para obtener la ER. ¿Qué debe cambiar respecto al procedimiento anterior?

Ejercicio: desarrollar un AF que reconozca el lenguaje de los números decimales divisibles por 3. Utilice el método dado para obtener la ER.

Ejercicio: Sugerir un algoritmo para determinar si dos ER son equivalentes. Ayuda: Utilizar la teoría de homomorfismos sobre autómatas.

El teorema de Kleene demuestra que tanto los Autómatas finitos como las expresiones regulares son modelos formales de la misma potencia, aunque sus especificaciones sean diferentes. Esta diferencia se debe a que corresponden a distintos tipos conceptuales de especificación de lenguajes. Las expresiones regulares corresponden al modelo denotacional y los Autómatas finitos al modelo operacional. El primero es estático, denota un lenguaje a través de una interpretación teórica de sus operadores. El segundo, modelo operacional, es dinámico, simula el comportamiento de un dispositivo describiendo una secuencia de estados, es decir tiene una evolución dinámica.

1.5.3 Propiedades de los Lenguajes Regulares

Al iniciar nuestro estudio de los AF hemos visto que la clase de lenguajes aceptados por AF deterministas es la misma que la clase de lenguajes aceptados por AF no deterministas. Ahora veremos otras propiedades de la clase de lenguajes aceptados por AF, las obtenidas por *clausura bajo ciertas operaciones*, que nos permitirán una mayor comprensión del concepto de lenguajes regulares. El estudio de dichas propiedades es importante por las siguientes razones:

brindan herramientas para la construcción y simulación de AF.

esclarecen aún mas el nexo entre los autómatas finitos y las expresiones regulares.

muestra que los LR son estables ¹ (ver cuadro comparativo de final de capítulo).

ayudan a identificar el tipo de un lenguaje.

simplifica la demostración de resultados teóricos.

TEOREMA 1.5.2 La clase de los lenguajes aceptados por AF, es decir la clase de los lenguajes regulares, es cerrada bajo: unión, concatenación, estrella de Kleene, complemento e intersección. ■

Demostración 7 Ya se ha demostrado que el teorema es cierto para las operaciones de unión, concatenación y estrella de Kleene. Cuando se consideró la primer parte del Teorema del Kleene construimos autómatas finitos que representaban el resultado de aplicar dichas operaciones a otros autómatas finitos reconociendo lenguajes regulares.

Veamos que tambien vale para la operación de complemento. Sea $M = (S, \Sigma, \delta, s, F)$ un autómata finito *determinista*. El lenguaje complemento $\Sigma^* - L(M)$ es aceptado por el siguiente autómata finito: $M' = (S, \Sigma, \delta, s, S - F)$. Es decir, M' es idéntico a M salvo por sus estados finales que están intercambiados.

Finalmente solo resta demostrar que los autómatas finitos son cerrados bajo intersección. Sean L_1 y L_2 dos lenguajes aceptados por dos autómatas finitos M_1 y M_2 , entonces, el lenguaje L , intersección de L_1 y L_2 , es $\Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$, es decir, el complemento de la unión de los complementos de L_1 y L_2 (por leyes de De Morgan). Por lo tanto, para construir el autómata finito de la intersección de L_1 y L_2 , construimos primero el autómata para el complemento de L_1 y L_2 , luego el autómata de la unión, hallamos el equivalente determinista y finalmente complementamos este último autómata. □

Las demostraciones dadas para las operaciones de unión e intersección son simples de entender y sencillas en cuanto a su construcción. Lamentablemente no constituyen buenas herramientas cuando se desea construir efectivamente un nuevo autómata que reconozca un lenguaje que representa la unión o intersección de otros previamente conocidos.

Algoritmo: (Unión)

Entrada: dos AF deterministas M_1 y M_2 .

Salida: un AF determinista M que reconoce $L(M_1) \cup L(M_2)$.

¹En el siguiente sentido: al operar dos elementos de dicha clase, al menos para las operaciones que nos interesan en este curso, se obtiene un elemento de la misma clase.

Sean L_1 y L_2 dos lenguajes regulares aceptados por autómatas finitos M_1 y M_2 respectivamente. Queremos especificar un algoritmo que permita construir un AF que reconozca el lenguaje Unión de L_1 y L_2 .

Sean $M_1 = (S_1, \Sigma, \delta_1, s_1, F_1)$ y $M_2 = (S_2, \Sigma, \delta_2, s_2, F_2)$, sin pérdida de generalidad, podemos asumir que los conjuntos de estados S_1 y S_2 son disjuntos, si no lo fueran: renombramos los estados de alguno de ellos. Construimos un autómata finito M que acepte $L(M_1) \cup L(M_2)$, $M = (S, \Sigma, \delta, s, F)$, tal que:

$S = \{z_0, \dots, z_k\}$ donde $z_i = \langle x_m, y_n \rangle$ y $x_m \in S_1$ e $y_n \in S_2$, para $0 \leq i \leq k$, la relación de un par $\langle x_m, y_n \rangle$ con z_i quedará determinado al definir δ ,

$z_0 = \langle s_1, s_2 \rangle$ es el estado inicial del nuevo autómata,

$\delta(\langle x_m, y_n \rangle, a) = \langle x'_m, y'_n \rangle$ para $a \in \Sigma$, si $\delta_1(x_m, a) = x'_m$ y $\delta_2(y_n, a) = y'_n$

$F = \{z_i, \dots, z_k \mid \text{si } i \leq j \leq k, z_j = \langle x_m, y_n \rangle \text{ y o bien } x_m \in F_1 \text{ o } y_n \in F_2\}$

□

Comenzando en el estado z_0 , M calcula $\delta(z_h, a) = z_i$, para cada símbolo $a \in \Sigma$, siguiendo la definición general dada para δ . Dado $\delta(\langle x_m, y_n \rangle, a)$ se considera el comportamiento de los autómatas a unir: supongamos que en el primer autómata $\delta_1(x_m, a)$ conduce a un estado x'_m y que en el segundo autómata $\delta_2(y_n, a)$ conduce a un estado y'_n , entonces en el autómata unión el estado $\langle x_m, y_n \rangle$ tendrá un arco al estado $\langle x'_m, y'_n \rangle$ rotulado con el símbolo a . Se sigue con este proceso hasta que el nuevo autómata está completamente definido, es decir, para todos los estados y cada símbolo posible del alfabeto el autómata sabe como debe reaccionar.

Mediante este procedimiento, toda cadena aceptada en alguno de los autómatas a unir (que llegaba por alguna secuencia de estados a uno aceptador), en el nuevo autómata será conducido a través de una secuencia de estados que contiene como parte del nombre a los del autómata correspondiente. Además, como el último estado al que arriba contiene en su nombre alguno de los aceptadores en los autómatas de partida, por la forma en que hemos definido el conjunto de estados aceptadores del autómata unión, también será aceptador de la cadena examinada en el autómata unión.

Inversamente, si una cadena es aceptada en el autómata unión, debe haber pasado por una secuencia de estados $z_0 = \langle x_{m_0}, y_{n_0} \rangle, z_1 = \langle x_{m_1}, y_{n_1} \rangle, \dots, z_k = \langle x_{m_k}, y_{n_k} \rangle$. Como la cadena es aceptada debe darse que $z_k \in F$ y que o bien la secuencia $x_{m_0}, x_{m_1}, x_{m_k}$ o $y_{m_0}, y_{m_1}, y_{m_k}$ debe conducir a aceptar la cadena en alguno de los autómatas usados para la unión.

EJEMPLO 1.5.8 Sean los siguientes AF: $M_1 = (S_1, \Sigma, \delta_1, x_1, F_1)$ donde:

$$S_1 = \{x_1, x_2, x_3\}$$

$$\Sigma = \{a, b\}$$

$$F_1 = \{x_3\}$$

δ	a	b
x_1	x_2	x_1
x_2	x_3	x_1
x_3	x_3	x_3

que reconoce el lenguaje de las cadenas $w \in \{a, b\}^*$ tales que contienen la subcadena $(a + b)^*aa(a + b)^*$ y el autómata: $M_2 = (S_2, \Sigma, \delta_2, y_1, F_2)$ donde:

$$S_2 = \{y_1, y_2\}$$

$$\Sigma = \{a, b\}$$

$$F_2 = \{y_2\}$$

δ	a	b
y_1	y_1	y_2
y_2	y_1	y_2

que reconoce el lenguaje de las cadenas $w \in \{a, b\}^*$ que terminan en la letra b , es decir: $(a^* + b^*)b$.

Para construir el autómata unión pasaremos a considerar los estados del nuevo autómata en función de los anteriores:

$$z_1 = \langle x_1, y_1 \rangle$$

En z_1 : si se lee una a se puede optar por x_2 en M_1 o y_1 en M_2 . Luego debemos considerar un estado nuevo, $z_2 = \langle x_2, y_1 \rangle$

En z_1 : si se lee una b se puede optar por x_1 en M_1 o y_2 en M_2 . Luego debemos considerar un estado nuevo, $z_3 = \langle x_1, y_2 \rangle$

En z_2 : si se lee una a se puede optar por x_3 en M_1 o y_1 en M_2 . Luego debemos considerar un estado nuevo: $z_4 = \langle x_3, y_1 \rangle$

En z_2 : si se lee una b se puede optar por x_1 en M_1 o y_2 en M_2 , z_3 .

En z_3 : si se lee una a se puede optar por x_2 en M_1 o y_1 en M_2 , z_2 .

En z_3 : si se lee una b se puede optar por x_1 en M_1 o y_2 en M_2 , z_3 .

En z_4 : si se lee una a se puede optar por x_3 en M_1 o y_1 en M_2 , z_4 .

En z_4 : si se lee una b se puede optar por x_3 en M_1 o y_2 en M_2 . Luego debemos considerar un estado nuevo: $z_5 = \langle x_3, y_2 \rangle$.

En z_5 : si se lee una a se puede optar por x_3 en M_1 o y_1 en M_2 , z_4 .

En z_5 : si se lee una b se puede optar por x_3 en M_1 o y_2 en M_2 , z_5 .

Luego el nuevo autómata queda completado a través de la siguiente especificación:

$M = (S, \Sigma, \delta, z_1, F)$ donde:

$$S_2 = \{z_1, z_2, z_3, z_4, z_5\}$$

$$\Sigma = \{a, b\}$$

$$F_2 = \{z_3, z_4, z_5\}$$

δ	a	b
z_1	z_2	z_3
z_2	z_4	z_3
z_3	z_2	z_3
z_4	z_4	z_5
z_5	z_4	z_5

□

Ejercicio: ¿Es un automata finito minimal?

Algoritmo: (Intersección)

Entrada: dos AF deterministas M_1 y M_2 .

Salida: un AF determinista M que reconoce $L(M_1) \cap L(M_2)$.

Sean L_1 y L_2 dos lenguajes regulares aceptados por autómatas finitos M_1 y M_2 respectivamente. Queremos especificar un algoritmo que permita construir un AF que reconozca el lenguaje Intersección de L_1 y L_2 .

Sean $M_1 = (S_1, \Sigma, \delta_1, s_1, F_1)$ y $M_2 = (S_2, \Sigma, \delta_2, s_2, F_2)$, sin pérdida de generalidad, podemos asumir que los conjuntos de estados S_1 y S_2 son disjuntos, si no lo fueran: renombramos los estados de alguno de ellos. Construimos un autómata finito M que acepte $L(M_1) \cap L(M_2)$. $M = (S, \Sigma, \delta, s, F)$, y:

$S = \{z_0, \dots, z_k\}$ donde $z_i = \langle x_m, y_n \rangle$ y $x_m \in S_1$ e $y_n \in S_2$, para $0 \leq i \leq k$, la relación de un par $\langle x_m, y_n \rangle$ con z_i quedará determinado al definir δ ,

$z_0 = \langle s_1, s_2 \rangle$ es el estado inicial del nuevo autómata,

$\delta(\langle x_m, y_n \rangle, a) = \langle x'_m, y'_n \rangle$ para $a \in \Sigma$, si $\delta_1(x_m, a) = x'_m$ y $\delta_2(y_n, a) = y'_n$

$F = \{z_i, \dots, z_k \mid \text{si } i \leq j \leq k, z_j = \langle x_m, y_n \rangle \text{ y simultáneamente } x_m \in F_1 \text{ e } y_n \in F_2\}$

□

Comenzando en el estado z_0 , M calcula $\delta(z_h, a) = z_i$, para cada símbolo $a \in \Sigma$, siguiendo la definición general dada para δ . Dado $\delta(\langle x_m, y_n \rangle, a)$ se considera el comportamiento de los autómatas a intersectar: supongamos que en el primer autómata $\delta_1(x_m, a)$ conduce a un estado x'_m y que en el segundo autómata $\delta_2(y_n, a)$ conduce a un estado y'_n , entonces en el autómata intersección el estado $\langle x_m, y_n \rangle$ tendrá un arco al estado $\langle x'_m, y'_n \rangle$ rotulado con el símbolo a . Se sigue con este proceso hasta que el nuevo autómata está completamente definido, es decir, para todos los estados y cada símbolo posible del alfabeto el autómata sabe como debe reaccionar.

Mediante este procedimiento, toda cadena aceptada en ambos autómatas a intersectar (que llegaba por alguna secuencia de estados a uno aceptador), en el nuevo autómata será conducido a través de una secuencia de estados que contiene como nombre de los automatas correspondientes. Además, como el último estado al que arriba contiene en su nombre ambos aceptadores en los autómatas de partida, por la forma en que hemos definido el conjunto de estados aceptadores del autómata intersección, también será aceptador de la cadena examinada en el autómata intersección.

Inversamente, si una cadena es aceptada en el autómata intersección, debe haber pasado por una secuencia de estados $z_0 = \langle x_{m_0}, y_{n_0} \rangle, z_1 = \langle x_{m_1}, y_{n_1} \rangle, \dots, z_k = \langle x_{m_k}, y_{n_k} \rangle$. Como la cadena es aceptada debe darse que $z_k \in F$ y que tanto la secuencia $x_{m_0}, x_{m_1}, x_{m_k}$ como $y_{m_0}, y_{m_1}, y_{m_k}$ conducen a aceptar la cadena en M_1 y en M_2 respectivamente.

EJEMPLO 1.5.9 Sean los siguientes AF:

$$M_1 = (S_1, \Sigma, \delta_1, x_1, F_1)$$

$$S_1 = \{x_1, x_2, x_3\}$$

$$\Sigma = \{a, b\}$$

$$F_1 = \{x_1, x_2\}$$

δ	a	b
x_1	x_2	x_1
x_2	x_3	x_1
x_3	x_3	x_3

que reconoce el lenguaje de las cadenas $w \in \{a, b\}^*$ tales que no contienen la subcadena aa , es decir $\Sigma - \{(a+b)^*aa(a+b)^*\}$, y el autómata

$$M_2 = (S_2, \Sigma, \delta_2, y_1, F_2)$$

$$S_2 = \{y_1, y_2\}$$

$$\Sigma = \{a, b\}$$

$$F_2 = \{y_2\}$$

δ	a	b
y_1	y_2	y_1
y_2	y_1	y_2

que reconoce el lenguaje de las cadenas $w \in \{a, b\}^*$ con un número impar de letras a es decir: $b^*a(b^*ab^*ab^*)^*$.

Para construir el autómata intersección utilizaremos la misma estrategia que para el autómata unión pero cambiaremos la designación de los estados finales a aquellos $z_j = \langle x_{m_j}, y_{n_j} \rangle$ tales que $x_{m_j} \in F_1$ y $y_{n_j} \in F_2$:

$$z_1 = \langle x_1, y_1 \rangle$$

En z_1 : si se lee una a se puede optar por x_2 en M_1 o y_2 en M_2 . Luego debemos considerar un estado nuevo, $z_4 = \langle x_2, y_2 \rangle$

En z_1 : si se lee una b se puede optar por x_1 en M_1 o y_1 en M_2 , z_1 .

En z_2 : si se lee una a se puede optar por x_2 en M_1 o y_1 en M_2 . Luego debemos considerar un estado nuevo: $z_3 = \langle x_2, y_1 \rangle$

En z_2 : si se lee una b se puede optar por x_1 en M_1 o y_2 en M_2 , Luego debemos considerar un estado nuevo: $z_2 = \langle x_1, y_2 \rangle$

En z_3 : si se lee una a se puede optar por x_3 en M_1 o y_2 en M_2 . Luego debemos considerar un estado nuevo: $z_6 = \langle x_3, y_2 \rangle$.

En z_3 : si se lee una b se puede optar por x_1 en M_1 o y_1 en M_2 , z_1 .

En z_4 : si se lee una a se puede optar por x_3 en M_1 o y_1 en M_2 . Luego debemos considerar un estado nuevo: $z_5 = \langle x_3, y_1 \rangle$.

En z_4 : si se lee una b se puede optar por x_1 en M_1 o y_2 en M_2 , z_2 .

En z_5 : si se lee una a se puede optar por x_3 en M_1 o y_2 en M_2 , z_6 .

En z_5 : si se lee una b se puede optar por x_3 en M_1 o y_1 en M_2 , z_5 .

En z_6 : si se lee una a se puede optar por x_3 en M_1 o y_1 en M_2 , z_5 .

En z_6 : si se lee una b se puede optar por x_3 en M_1 o y_2 en M_2 , z_6 .

Luego el nuevo autómata queda definido a través de la siguiente especificación:

$$M = (S, \Sigma, \delta, z_1, F)$$

$$S_2 = \{z_1, z_2, z_3, z_4, z_5, z_6\}$$

$$\Sigma = \{a, b\}$$

$$F_2 = \{z_2, z_4\}$$

δ	a	b
z_1	z_4	z_1
z_2	z_3	z_2
z_3	z_6	z_1
z_4	z_5	z_2
z_5	z_6	z_5
z_6	z_5	z_6

□

1.6 Gramáticas

Suponga que Ud. cuenta con un procesador capaz de reconocer una oración válida del lenguaje Español. En tal caso dada la oración “El cielo es azul” podría determinar que ésta es sintácticamente correcta. También podría determinar que “azul El es cielo” no es sintácticamente correcta. Es decir que puede en forma inmediata, dada una oración, establecer si está bien formada según un conjunto de reglas estudiadas. En esos casos está actuando como un reconocedor. Inspirados en esta situación hipotética definiremos el reconocedor de un lenguaje como un dispositivo que acepta sentencias válidas de un lenguaje. En tal sentido, los autómatas finitos son reconocedores de los lenguajes regulares. Los humanos no solo somos capaces de reconocer la validez de las oraciones sino que habitualmente hablamos y escribimos, es decir, estamos actuando como generadores de lenguajes.

En esta sección estudiaremos uno de los tipos de generadores de lenguajes formales: las gramáticas estructuradas por frases. Estos dispositivos permiten comenzar el proceso de generación a partir de un símbolo designado con antelación para tal fin y su operación está limitada por un conjunto de reglas. La teoría que describe los generadores de lenguajes, es decir las gramáticas, complementa a la de autómatas. Ambos son necesarios en la especificación y análisis de los lenguajes de computación.

DEFINICIÓN 1.6.1 Definiremos *gramática estructurada por frases (GEF)*, como una cuádrupla (V_n, V_t, S, P) donde:

V_n es un conjunto finito de *símbolos no terminales* o *símbolos auxiliares*.

V_t es un conjunto finito de *símbolos terminales*.

S es el *símbolo inicial*.

P conjunto finito de *reglas de producción* de la forma $\alpha \rightarrow \beta$ donde $\alpha \in (V_n \cup V_t)^* V_n (V_n \cup V_t)^*$ y $\beta \in (V_n \cup V_t)^*$. Las reglas de producción nos permiten generar las denominadas “cadenas” o “palabras”, también son llamadas *reglas de reescritura*.



De aquí en más, siempre que hagamos referencia a una gramática asumiremos que: $V_n \neq \emptyset$, $V_t \neq \emptyset$ y $V_n \cap V_t = \emptyset$ aunque a manera de simplificación no lo repetamos en cada una de las definiciones de gramáticas que daremos. También es importante aclarar que sobre el caso de una regla $\beta = \lambda$ nos explayaremos más adelante. Por el momento alcanza con comprender que una regla del tipo $\alpha \rightarrow \lambda$ significa que toda la cadena α es reemplazada por la cadena vacía λ , lo cual en terminos prácticos e intuitivos puede interpretarse como borrar α .

EJEMPLO 1.6.1 sea la gramática

$$\begin{aligned} G &= (V_n, V_t, S, P) \\ V_n &= \{S\} \\ V_t &= \{0, 1\} \\ P &= \{S \rightarrow 0S, S \rightarrow 1\} \end{aligned}$$

Una pregunta que típicamente deberemos responder es ¿Qué lenguaje genera una gramática dada? Para encontrar la respuesta podemos inspeccionar algunas cadenas que genera. Si en la gramática recién mencionada se observa el conjunto de reglas de producción puede notarse que la gramática está en condiciones de generar la cadena “1” utilizando la segunda regla de reescritura y transformando el símbolo inicial, S , en el símbolo terminal antes mencionado. En cambio, si se comienza la secuencia de generación de una cadena reescribiendo el símbolo inicial por medio de la primer regla se obtiene la cadena “0S”. Esta cadena aún no está constituida por terminales solamente. Sus símbolos no terminales pueden ser reescritos utilizando las reglas de P . Si se deseara, utilizando la última regla de reescritura podría reemplazarse el no terminal S por el terminal “1” arribando a la cadena “01”. Análogamente, si se usa dos veces la primera regla de reescritura antes de usar la segunda, se obtiene la cadena “001”. En general, puede demostrarse por inducción que el lenguaje generado por la gramática dada es el conjunto de cadenas con una cantidad arbitraria de ceros y terminadas en 1, es decir 0^*1 . $\square$

Cada uno de los pasos en el analisis previo es denominado “una *derivación*”. El concepto de derivación es de suma importancia, en este contexto representa los pasos del proceso por el cual se obtiene una cadena a partir de las reglas de la gramática. Formalmente, se trata de una relación entre cadenas de símbolos terminales y/o no terminales. Dos cadenas α_1, α_2 estan relacionadas a traves de una derivación si existe una regla de producción, en la gramática utilizada, para reescribir una parte de α_1 en otra parte de α_2 . Es útil distinguir dos tipos de relaciones de derivación:

deriva directamente, que se representa mediante el símbolo $\xrightarrow{G}$, significando que una cadena se obtiene en un paso utilizando las reglas de la gramática G y

deriva, que se representa el símbolo $\xrightarrow{G^+}$, significando que una cadena se obtiene en uno o más pasos utilizando las reglas de la gramática G .

A continuación definiremos mas formalmente ambas relaciones, lo cual nos permitirá definir con precisión que es el lenguaje generado por una gramática.

DEFINICIÓN 1.6.2 Sea $G = (V_n, V_t, S, P)$ una gramática y $\varphi, \mu \in (V_t \cup V_n)^*$, diremos que $\mu\alpha\varphi$ *deriva directamente* a $\mu\beta\varphi$ en G , lo escribiremos como $\mu\alpha\varphi \xrightarrow{G} \mu\beta\varphi$, si existe una regla de producción $\alpha \rightarrow \beta$ en P . $\blacksquare$

DEFINICIÓN 1.6.3 Dadas dos cadenas α, β se dice que α *deriva a* β según la gramática G , notado $\alpha \xrightarrow{G^+} \beta$, si β puede obtenerse por aplicación de reglas de producción de G a partir de α , es decir, $\alpha \xrightarrow{G} \gamma_1, \gamma_1 \xrightarrow{G} \gamma_2, \dots, \gamma_j \xrightarrow{G} \gamma_k, \gamma_k \xrightarrow{G} \beta$. ■

DEFINICIÓN 1.6.4 Sea una gramática $G = (V_n, V_t, S, P)$, $L(G) = \{w \in V_t^* | S \xrightarrow{G^+} w\}$ es el *lenguaje generado por* G . ■

Es decir, L es el conjunto de todas las cadenas de terminales generadas (derivadas) a partir del símbolo inicial.

EJEMPLO 1.6.2 En la gramática anterior tenemos que $000S \xrightarrow{G} 0001$, gracias a la producción $S \rightarrow 1$, también se puede verificar que $0S \xrightarrow{G^+} 000001$. El análisis posterior a la especificación de la gramática permite concluir que $L(G) = 00^*1$. □

1.6.1 Gramáticas Regulares

La definición de gramática estructurada por frases que hemos considerado previamente permite la consideración de varios casos particulares. Existe una clasificación de gramáticas basada en restricciones impuestas a la naturaleza de las producciones.

DEFINICIÓN 1.6.5 Una *gramática regular* (GR) es una cuádrupla (V_n, V_t, S, P) donde

V_n conjunto de símbolos no terminales.

V_t conjunto de símbolos terminales.

S es el símbolo inicial.

P es un conjunto finito de reglas de producción de la forma $\alpha \rightarrow \beta$ tales que:

α es un solo no terminal, $\alpha \in V_n$.

β es un solo terminal ó es un terminal concatenado con un no terminal;

es decir ², $\beta \in V_t$ o $\beta \in V_t V_n$, donde $a \in V_t, B \in V_n$

■

Observación: las gramáticas regulares así definidas son denominadas *Regulares a Derecha*. Análogamente pueden definirse las *Gramáticas Regulares a Izquierda* reemplazando en la definición a $\beta \in V_t V_n$ por $\beta \in V_n V_t$.

EJEMPLO 1.6.3 Diseñar una gramática que genere las cadenas pertenecientes a $\{a, b\}^+$ que comienzan con a , es decir: $a(a+b)^*$.

$$G = (V_n, V_t, S, P)$$

$$V_t = \{a, b\}$$

$$V_n = \{S, B\}$$

$$P = \{S \rightarrow a, S \rightarrow aB, B \rightarrow bB, B \rightarrow aB, B \rightarrow a, B \rightarrow b\}$$

Supongamos que deseamos saber si $abbaba \in L(G)$. Podemos averiguarlo verificando si la cadena es generable a través de la gramática:

$$S \xrightarrow{G} aB \xrightarrow{G} abB \xrightarrow{G} abbB \xrightarrow{G} abbaB \xrightarrow{G} abbabB \xrightarrow{G} abbaba$$

Luego, como $S \xrightarrow{G^+} abbaba$ podemos asegurar que $abbaba \in L(G)$. □

²De aquí en adelante adoptaremos la convención de notar con $V_t \bullet V_n$ o simplemente $V_t V_n$ al conjunto $\{\omega | \omega = x \bullet y \text{ para algún } x \in V_t \text{ e } y \in V_n\}$ en forma análoga a la concatenación de lenguajes.

EJEMPLO 1.6.4 Diseñar una gramática que genere las cadenas pertenecientes a $\{a, b\}^+$ que contienen tres b 's consecutivas, es decir el lenguaje denotado por $(a + b)^*bbb(a + b)^*$.

$$G = (V_n, V_t, S, P)$$

$$V_t = \{a, b\}$$

$$V_n = \{S, T_1, T_2, T_3\}$$

$$P = \{S \rightarrow aS; S \rightarrow bS;$$

$$S \rightarrow bT_1; T_1 \rightarrow bT_2; T_2 \rightarrow bT_3; T_2 \rightarrow b;$$

$$T_3 \rightarrow aT_3; T_3 \rightarrow bT_3; T_3 \rightarrow a; T_3 \rightarrow b\}$$

Una derivación en esa gramática es:

$$S \xrightarrow{G} aS \xrightarrow{G} abS \xrightarrow{G} abbS \xrightarrow{G} abbaS \xrightarrow{G} abbabT_1 \xrightarrow{G} abbabbT_2 \xrightarrow{G} abbabbb$$

es decir: $S \xrightarrow{G^+} abbabbb$

□

EJEMPLO 1.6.5 Diseñar una gramática que genere el lenguaje de los identificadores válidos de Pascal sin límites en la longitud.

$$G = (V_n, V_t, S_0, P)$$

$$V_n = \{S_0, S_1\}$$

$$V_t = \{a \dots z\} \cup \{A \dots Z\} \cup \{0 \dots 9\}$$

$$P = \{ \begin{array}{l} S_0 \rightarrow a|b|c|\dots|z \\ S_0 \rightarrow A|B|C|\dots|Z \\ S_0 \rightarrow aS_1|bS_1|cS_1|\dots|zS_1 \\ S_0 \rightarrow AS_1|BS_1|CS_1|\dots|ZS_1 \\ S_1 \rightarrow aS_1|bS_1|cS_1|\dots|zS_1 \\ S_1 \rightarrow AS_1|BS_1|CS_1|\dots|ZS_1 \\ S_1 \rightarrow 0S_1|1S_1|2S_1|\dots|9S_1 \\ S_1 \rightarrow 0|\dots|9 \\ S_1 \rightarrow a|\dots|z \\ S_1 \rightarrow A|\dots|Z \end{array} \}$$

Nota: Se ha utilizado el símbolo ‘|’ para marcar las alternativas; la definición estricta de gramáticas exige una regla de producción por cada alternativa de reescritura que figure a la derecha de las reglas. □

En secciones anteriores hemos examinado un resultado teórico muy útil, el teorema de Kleene. Dicho resultado nos permite conocer la conexión entre los autómatas finitos y la clase de lenguajes denominados regulares. Posteriormente hemos introducido la noción de gramática regular y hemos mostrado cómo dichas gramáticas nos permiten generar lenguajes. Con estos conocimientos en mano surge el interrogante de cuál es la clase de lenguajes que este tipo de gramáticas permite generar y si es posible relacionarlos a aquellos denotados por ER y reconocidos por AF. Dedicaremos el resto del capítulo a estudiar tal relación.

1.6.2 Equivalencia entre Gramáticas Regulares y Autómatas Finitos

A continuación, exploraremos un resultado que es, a priori, sorprendente. Dos formalismos con distintas características externas poseen una relación muy fuerte en cuanto al alcance de los lenguajes que procesan. Veremos como los autómatas finitos y las gramáticas regulares cubren diversos aspectos de los denominados Lenguajes Regulares y más aún, exploraremos una demostración que provee métodos para obtener la gramática dado el autómata y viceversa.

TEOREMA 1.6.1 Sea G una gramática regular. $L(G)$ es un lenguaje generado por G si y solo si existe un autómata finito M que reconoce $L(M)$, tal que $L(M) = L(G)$. ■

Demostración 8 Consideremos la primer mitad del enunciado del teorema “Sea $G = (V_n, V_t, S, P)$ una gramática regular entonces existe un autómata finito M tal que si $x \in L(M)$ entonces $x \in L(G)$ ”. Consideremos $M = (K, V_t, \delta, S, F)$. Notese que el alfabeto del autómata finito es el conjunto de terminales de la gramática y que el estado inicial del autómata finito es el símbolo inicial de la gramática. Dada la gramática G especificaremos el resto del autómata finito no determinista, M , de la siguiente forma:

$K = V_n \cup \{A, R\}$ donde $A, R \notin V_n$.

Si P tiene la producción $S \rightarrow \lambda$, entonces $F = \{A, S\}$ sino $F = \{A\}$.

Al definir δ debemos considerar los siguientes casos:

$\delta(B, a) \supseteq \{A\}$, si ‘ $B \rightarrow a$ ’ $\in P$; $a \in V_t$; $B \in V_n$.

$\delta(B, a) \supseteq \{C\}$, si ‘ $B \rightarrow aC$ ’ $\in P$; $a \in V_t$; $B, C \in V_n$.

$\delta(B, a) \supseteq \{R\}$, $R \notin F$, para todo par $(B, a) \in S \times \Sigma$ no considerado en la definición del autómata a través de las reglas anteriores.

Observación: con el último paso de este procedimiento se evita que el autómata finito quede incompletamente especificado. Para esto se rechaza el resto de las cadenas que no han sido consideradas en la gramática. También puede definir AF no deterministas. En tal caso se puede usar el procedimiento visto en la sección 1.3.6 para obtener un autómata determinista equivalente si fuera necesario.

Consideremos la otra mitad del enunciado del teorema: “Dado un autómata finito M existe una gramática regular G tal que si $x \in L(G)$ entonces $x \in L(M)$ ”. Sin pérdida de generalidad podemos considerar un AFND $M = (S, \Sigma, \delta, s_0, F)$. Definamos una gramática $G = (S, \Sigma, s_0, P)$ donde P está formada por

$B \rightarrow aC$, si $\delta(B, a) \supseteq \{C\}$

$B \rightarrow a$ si $\delta(B, a) \supseteq \{C\}$ y $C \in F$.

Agregar $S \rightarrow \lambda$ si $s_0 \in F$.

De esta forma proveemos una manera de definir una gramática a partir del AF inicial que permite generar un lenguaje relacionado, en alguna medida, al reconocido por el autómata inicial. Veamos que los lenguajes involucrados en los procedimientos de traducción son equivalentes.

1. También podemos concluir que si $x \in L(G)$, entonces $x \in L(M)$:

Si $\lambda \in L(M)$, entonces $S \in F$; entonces $S \rightarrow \lambda \in P$ y $\lambda \in L(G)$. En el caso general, si $x \in L(M)$, entonces existe una secuencia de estados $S, A_1, A_2 \dots A_{n-1}, A$ tal que:

$\delta(S, a_1) \supseteq \{A_1\}$, $\delta(A_1, a_2) \supseteq \{A_2\}, \dots, \delta(A_{n-1}, a_n) \supseteq \{A\}$; entonces P contiene reglas de la forma: $S \rightarrow a_1 A_1, A_1 \rightarrow a_2 A_2, \dots, A_{n-1} \rightarrow a_n$. Entonces $S \xrightarrow{G} a_1 A_1 \xrightarrow{G} a_1 a_2 A_2 \dots \xrightarrow{G^+} a_1 a_2 \dots a_n$ es una secuencia de derivaciones en G , como $S \xrightarrow{G^*} a_1 a_2 \dots a_n$ entonces $x \in L(G)$. Luego es cierto para toda cadena x que si $x \in L(M)$ entonces $x \in L(G)$. CQD.

2. Veamos que si $x \in L(M)$ entonces $x \in L(G)$:

En particular si $\lambda \in L(G)$ entonces $S \in F$ luego $\lambda \in L(M)$. En el caso general, sea $x = a_1 a_2 \dots a_n$ tal que $x \in L(G), n \geq 1$. Entonces $S \xrightarrow{G} a_1 A_1 \xrightarrow{G} a_1 a_2 A_2 \dots \xrightarrow{G^+} a_1 a_2 a_3 \dots a_{n-1} A_{n-1} \xrightarrow{G^+} a_1 \dots a_n$ para una secuencia de no terminales $A_1, A_2 \dots A_{n-1}$.

De la definición de δ tenemos que $\delta(S, a_1) \supseteq \{A_1\}$, $\delta(A_1, a_2) \supseteq \{A_2\} \dots$ y $\delta(A_{n-1}, a_n) \supseteq \{A\}$. Entonces $x \in L(M)$, pues $\delta(S, x) \supseteq \{A\}$ y $A \in F$. Luego es cierto para toda cadena x que si $x \in L(G)$ entonces $x \in L(M)$. CQD. $\square$

Este resultado complementa el teorema de Kleene ya que provee una conexión indirecta entre las gramáticas regulares y los lenguajes regulares denotados por expresiones regulares a través de autómatas finitos (ver gráfico 1.6).

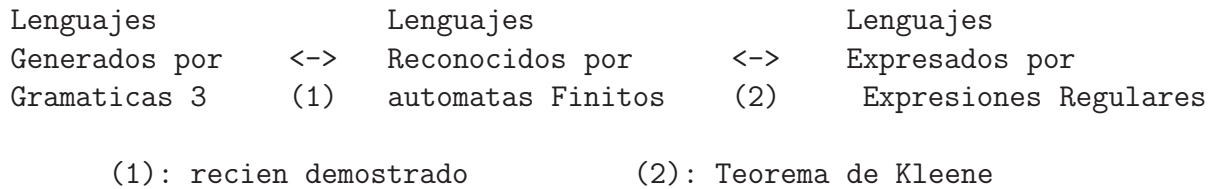


Figura 1.6: Relación entre los distintos tipos de lenguajes estudiados

Esta interconexión de conceptos es muy importante, ahora conocemos con mayor precisión la relación existente entre los conceptos que hemos estudiado y permite saber que dado cualquiera de ellos puedo obtener el mismo objeto desde otra perspectiva. Luego, si lo único que se tiene es el autómata finito que reconoce un lenguaje regular L sabemos que existe una gramática regular que genera dicho lenguaje. Mas aún, dado el caracter constructivo de las demostraciones consideradas, las mismas permiten obtener una gramática concreta que genere dicho lenguaje. Veamos a continuación algunos ejemplos que muestran como aplicar en forma constructiva dichas demostraciones.

EJEMPLO 1.6.6 Considerese la siguiente gramática que permite generar la palabra reservada *program* del lenguaje de programación Pascal:

$$\begin{aligned}
 G &= (V_n, V_t, S, P) \\
 V_n &= \{S, S_1, S_2, S_3, S_4, S_5, S_6\} \\
 V_t &= \{p, r, o, g, a, m\} \\
 P &= \{S \rightarrow pS_1, S_1 \rightarrow rS_2, S_2 \rightarrow oS_3, S_3 \rightarrow gS_4, S_4 \rightarrow rS_5, S_5 \rightarrow aS_6, S_6 \rightarrow m\}
 \end{aligned}$$

Podemos construir el autómata finito a partir de G definiendo $M = (K, \Sigma, \delta, s_0, F)$ tal que

$$\begin{aligned}
 K &= V_n \cup \{A\} \quad \Sigma = V_t \\
 s_0 &= S \quad F = \{A\}
 \end{aligned}$$

La demostración del teorema sugiere el siguiente autómata

δ	p	r	o	g	a	m	$\Sigma - \{p, r, o, g, a, m\}$
S_0	S_1	R	R	R	R	R	R
S_1	R	S_2	R	R	R	R	R
S_2	R	R	S_3	R	R	R	R
S_3	R	R	R	S_4	R	R	R
S_4	R	S_5	R	R	R	R	R
S_5	R	R	R	R	S_6	R	R
S_6	R	R	R	R	R	A	R
R	R	R	R	R	R	R	R
A	R	R	R	R	R	R	R

□

Si bien la definición dada de gramáticas regulares dice que una producción es de la forma $\alpha \rightarrow \beta$, con $\alpha \in V_n$ y $\beta \in V_t$ ó $\beta \in (V_t \bullet V_n)$, se puede probar que esta definición es equivalente a una en la cual se permite que $\beta \in V_t^+$ ó $\beta \in (V_t^+ \bullet V_n)$. Por lo tanto podríamos haber definido una gramática con una sola producción: $P = \{S \rightarrow \text{program}\}$.

Ejercicio: ¿Cuál es la expresión regular que representa la cadena “program”? ¿Y de cualquier palabra reservada del lenguaje Pascal?

EJEMPLO 1.6.7 Desarrollar a partir del autómata finito que reconoce el lenguaje $L = \{w | w \text{ tiene } 3k + 1 \text{ b's, } k \geq 0\}$ la gramática regular que lo genera (ver ejemplo de sección 1.5.2). Consideremos entonces el autómata $M = (S, \Sigma, \delta, s_0, F)$ donde:

$$S = \{s_0, s_1, s_2\} \quad \Sigma = \{a, b\} \quad F = \{s_1\}$$

δ	a	b
s_0	s_0	s_1
s_1	s_1	s_2
s_2	s_2	s_0

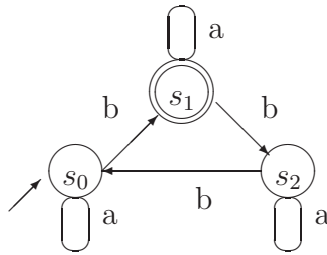


Figura 1.7: Autómata para reconocer una secuencia de $3k+1$ b's

Construyamos la gramática G según el teorema anterior:

$$G = (S, \Sigma, s_0, P)$$

$$V_n = \{s_0, s_1, s_2\}$$

$$V_t = \{a, b\}$$

$$S = s_0$$

$$P = \{s_0 \rightarrow as_0, s_0 \rightarrow b, s_0 \rightarrow bs_1, s_1 \rightarrow as_1, s_1 \rightarrow a, s_1 \rightarrow bs_2, s_2 \rightarrow as_2, s_2 \rightarrow bs_0\}$$

□

1.7 Pruebas sobre lenguajes regulares

A través de este capítulo hemos podido comprobar que los lenguajes regulares poseen propiedades muy especiales. Sabemos que son representables mediante expresiones regulares, que son reconocibles mediante los autómatas finitos y son generables por gramáticas regulares. Todas estas nociones poseen una serie de características deseables como simplicidad, facilidad para su formalización y estudio matemático, etc.³ Luego, es deseable poder demostrar que un lenguaje dado es regular, dado que entre otras ventajas sabemos que bastará para su procesamiento con el más simple de los medios de computación que veremos en este curso: un autómata finito. También es útil poder asegurarnos que un lenguaje no es regular. Si no pudieramos demostrar que es regular y para procesarlo utilizáramos un autómata más sofisticado que un autómata finito, podríamos estar cometiendo el error de desperdiciar esfuerzo en un lenguaje para el cual existe un tratamiento más simple. En esta sección consideraremos herramientas que nos permiten determinar si un lenguaje es regular o no.

1.7.1 Demostración de que un lenguaje es regular

Hemos visto que los lenguajes regulares son: a) especificados por expresiones regulares, b) aceptados por autómatas finitos, c) cerrados bajo unión, concatenación, estrella de Kleene, complemento e intersección y d) generados por gramáticas regulares. Estos hechos pueden ser utilizados en forma individual o combinados con otras técnicas para probar que un lenguaje es regular.

EJEMPLO 1.7.1 Sean $\Sigma = \{a, b\}$ y $L \subseteq \Sigma^*$ el conjunto de todas las cadenas de longitud impar que contienen un número par de letras a , probar que L es regular.

Podemos comprobar que L es regular pues es la intersección de dos lenguajes regulares L_1 y L_2 . En efecto, si tomamos L_1 como el conjunto de cadenas de longitud impar representado por la ER $(a + b)((a + b)(a + b))^*$ y L_2 como el conjunto de cadenas con número par de letras a , es decir, $b^*(ab^*ab^*)^*$, L puede ser expresado como $L = L_1 \cap L_2$. Luego, L es regular. $\square$

1.7.2 Demostración de que un lenguaje no es regular

Hasta ahora vimos herramientas para probar que un lenguaje es regular, pero ¿cómo determinamos que un lenguaje no es regular?. El lector puede verificar por construcción que todo lenguaje finito es regular ya que puede construirse un autómata finito reconocedor. Luego, sea $L = \{a^i b^i | 0 \leq i \leq 3\}$, L es regular pues es finito. También hay lenguajes infinitos regulares como $L = \{a^* b^*\} = \{a^m b^n; m, n \geq 0\}$. Pero, no todo lenguaje infinito es regular, luego se plantea el problema de cómo reconocer los lenguajes que no son regulares. Por ejemplo, ¿ $L = \{a^n b^n | n \geq 0\}$ es regular? Intuitivamente podemos ver que no, pues es necesario considerar una cantidad infinita de posibilidades al momento de construir un autómata para reconocerlo. El siguiente teorema, presentado en [BHPS64] brinda un contexto más formal para tales pruebas.

TEOREMA 1.7.1 Si L es un lenguaje regular infinito, entonces existe un entero positivo k , dependiente del lenguaje, tal que para toda cadena $w \in L$ con $\text{long}(w) \geq k$, existen cadenas x, y, z con $w = xyz$, $0 < \text{long}(y) \leq k$ y $xy^n z \in L$, para todo $n \geq 0$. $\blacksquare$

³Naturalmente, estas características le serán más evidentes al lector cuando profundicemos nuestro estudio de gramáticas en los próximos capítulos.

Demostración 9 (Esquema) Podemos entender el enunciado del teorema apelando a la ya conocida relación entre los lenguajes regulares y los autómatas finitos. Ya que w consiste de al menos k símbolos, procesarla en un autómata finito de k estados requiere del uso de un arco mas de una vez. Luego, si el autómata finito considerado tiene k estados (la hipótesis del teorema), deberá utilizar al menos algún estado más de una vez para poder procesar la cadena de longitud k . Entonces una parte no nula, y , de la cadena considerada, w , es reconocida a través de un ciclo por el AF y entonces las cadenas que surgen de usar ese ciclo varias veces también serán reconocidas. Es decir, $\delta'(s_0, xyz) = \delta'(s_0, xy^n z) \in F$ y por lo tanto $xy^n z \in L$. $\square$

Este teorema es uno de la familia de los denominados “Pumping Theorem” que afirman la existencia de subcadenas que pueden ser insertadas en forma repetida y la cadena resultado pertenece al lenguaje de partida. Cabe destacarse que hemos mantenido el nombre del teorema en inglés, *pumping theorem*⁴, para facilitar su identificación en los libros de texto. El significado del verbo *pump* es “bombear”, una traducción no literal pero más adecuada al uso que aquí se le dará sería “repetir”.

EJEMPLO 1.7.2 Dado el lenguaje $L = \{a^n b^n | n > 0\}$. Probar que no es regular.

Supongamos que L es regular; sea k el entero del que habla el teorema. Tomemos la cadena $w = a^k b^k \in L$ (en este caso, $\text{long}(a^k b^k) = 2k > k$) y veamos que no es posible descomponer la cadena w en la forma xyz , tal que $xy^i z \in L$, para todo $i \geq 0$. Tenemos 3 casos posibles: y está formada sólo por letras a , y está formada sólo por letras b o y está formada por un número de letras a seguido de un número de letras b .

Caso 1: tomamos y enteramente formada por letras a , entonces:

$$\begin{aligned} x &= a^p \\ y &= a^q \\ z &= a^r b^s \text{ donde } s = p + q + r \text{ siendo } p, r \geq 0 \text{ y } q, s > 0. \text{ Por lo tanto} \\ xy^n z &= a^{(p+nq+r)} b^s \text{ para cada } n \geq 0, \text{ pero si } s = p + q + r \text{ luego } s \neq p + nq + r \\ &\text{salvo para } n = 1, \text{ luego } xy^n z \notin L \text{ si } n \neq 1 \end{aligned}$$

Caso 2: y contiene todas b

$$\begin{aligned} x &= a^p b^r \\ y &= b^q \\ z &= b^s \text{ donde } q > 0, s \geq 0 \text{ y } s + q + r = p \\ \text{Pero si } xy^n z &= a^p b^{(r+nq+s)} \text{ debería ser } s + n * q + r = p, \text{ pero nuevamente, esto} \\ &\text{vale sólo cuando } n = 1 \end{aligned}$$

Caso 3: y contiene a y b , entonces en $xy^n z$ hay ocurrencias de b que preceden las letras a y la cadena no mantiene la forma.

Por lo tanto, no hay forma de descomponer $w = xyz$ tal que $xy^n z$ pertenezca al lenguaje para todo valor de $n \geq 0$. Esto implica que L no es regular. CQD $\square$

⁴También citado como “pumping lemma”. Aquí preferimos la denominación de “Teorema” por que el resultado no es utilizado como base de resultados teoricos posteriores.

EJEMPLO 1.7.3 Probar que el lenguaje $L = \{a^p | p \text{ es primo}\}$ no es regular. Supongamos por el absurdo que L es regular y apliquemos el Pumping Theorem. En ese caso, existe un valor k tal que para toda cadena w tal que $\text{long}(w) \geq k$, $w = xyz$ y $xy^n z \in L$, para todo $n \geq 0$, donde $0 < \text{long}(y) \leq k$. Sea w una cadena perteneciente a L tal que $\text{long}(w) \geq k$, entonces, si queremos escribir $w = xyz$, tendremos que tomar la cadena y formada por un número de letras “a” mayor o igual que 1 (puesto que $y \neq \lambda$).

Sean $x = a^p$, $y = a^q$ y $z = a^r$ donde $p, r \geq 0, q > 0$. Por el Pumping Theorem, $xy^i z \in L$, para todo $i \geq 0$, esto es $a^{(p+nq+r)} \in L$ para cada $n \geq 0$. Pero esto no es posible, ya que si $n = p + 2q + r + 2$, entonces $p + nq + r = (q + 1) * (p + 2 * q + r)$ que es el producto de dos números naturales, mayores que 1, por lo tanto no es primo. El absurdo provino de suponer que L es regular, por lo tanto, L no es regular. CQD. $\square$

Otra aplicación del Pumping Theorem surge de combinarlo con el uso de propiedades de los lenguajes regulares para poder deducir el tipo de un lenguaje.

EJEMPLO 1.7.4 Sea el lenguaje $IGUALES \subset \{a, b\}^*$, formado por el conjunto de cadenas que tienen el mismo número de letras a y b . Supongamos que se desea determinar si el lenguaje $IGUALES$ es regular. Un modo de resolver el problema es pensar al lenguaje $\{a^n b^n | n > 0\}$ como la intersección de $\{a^* b^*\}$ con $IGUALES$. Es decir, $\{a^n b^n | n > 0\} = \{a^* b^*\} \cap IGUALES$. A partir de esta relación se puede utilizar una demostración, por el absurdo, de que el lenguaje $IGUALES$ no es regular como sigue. Con anterioridad hemos demostrado que los lenguajes regulares son cerrados bajo la propiedad de intersección. Luego si suponemos por el absurdo que $IGUALES$ es regular, al intersectarlo con otro lenguaje regular, $a^* b^*$ claramente lo es, deberíamos obtener un lenguaje regular. Pero como demostramos anteriormente mediante el uso del Pumping Theorem, $\{a^n b^n | n > 0\}$ no lo es. Luego, hemos llegado a un absurdo y éste provino de suponer a $IGUALES$ como regular. CQD. $\square$

Siguiendo una línea de razonamiento similar a la anterior, también puede utilizarse una combinación de propiedades con Pumping Theorem para simplificar el problema a resolver.

EJEMPLO 1.7.5 Sea el problema de determinar el tipo de un lenguaje C y sea $A = B\theta C$ donde θ es una operación binaria para la cual la clase de lenguajes a la que hipotéticamente pertenecen B y C es cerrada. Podemos reformular el problema inicial y reducir el problema de determinar el tipo de C al de demostrar el de A . Naturalmente, es conveniente utilizar esta estrategia cuando nos reporta una ganancia. Algunas situaciones en que es conveniente su uso son cuando el lenguaje A tiene una expresión más sencilla que la de C o cuando la determinación de la clase de lenguaje a la que pertenece requiere de un proceso más sencillo.

Una posible aplicación de esta estrategia podría haber sido si en el ejemplo anterior hubiéramos querido determinar el tipo de $IGUALES$ sin contar con la demostración de que $\{a^n b^n | n > 0\}$ no es regular. En tal caso habríamos reducido la determinación del tipo de $IGUALES$ al resultado de la determinación del tipo de $\{a^n b^n | n > 0\}$ ya que la aplicación del Pumping Theorem sobre este último es más sencilla. CQD. $\square$

1.8 Aplicaciones

En la presentación de los temas considerados en este capítulo hemos considerado ejemplos sencillos tendientes a ejemplificar dichos conceptos sin distraer al lector con las peculiaridades

que cada problema particular puede presentar. El proceder de tal manera es apropiado desde el punto de vista didáctico, pero se corre el riesgo de dejar en el lector la falsa sensación de que la utilidad de estas herramientas esta necesariamente circumscripta a problemas de índole teórica. Para atenuar o eliminar esta falsa conclusión dedicaremos una sección final en cada capítulo específicamente destinada a mostrar la utilidad práctica de dichas herramientas en relación con la resolución de problemas mediante el uso de computadoras. Naturalmente en ningún caso intentaremos dar una lista exhaustiva de las situaciones en las que estos formalismos tengan una aplicación exitosa. En su lugar comentaremos algunos casos de aplicación, dejando librado a la imaginación del lector y cursos posteriores la ampliación de esta enumeración de posibilidades.

Se ha sugerido a través del capítulo la utilidad de usar automatas finitos en relación al problema de reconocer palabras lo cual tiene una aplicación inmediata en la construcción de los programas denominados *compiladores*. Estos tienen a su cargo determinar si un programa escrito en determinado lenguaje de programación es correcto o no según la definición de dicho lenguaje. Como parte del problema se requiere el poder resolver el subproblema de determinar si cada cadena que forma parte del programa es una palabra aceptable en dicho lenguaje. El problema puede ser extendido facilmente al de reconocer una secuencia de palabras o un conjunto de palabras alternativas. El lector identificará en este capítulo los desarrollos teóricos mostrando que un automata finito provee un medio aceptable para ser usado en tales situaciones.

El interés por contar con un dispositivo capaz de reconocer las palabras que conforman un lenguaje no solo se hace presente cuando se diseña un compilador para un lenguaje de programación. En cualquier contexto en que se diseña una base de datos, esto es un medio de almacenar y recuperar información, existirá un interés por contar con un lenguaje para comunicarse con el sistema que administra dicha base de datos. En tal caso nuevamente surge la necesidad de contar con un medio para reconocer las palabras que conforman dicho lenguaje especializado en indicar al administrador de la base de datos que información se desea almacenar u obtener. Otra posible aplicación aparece en el contexto de traducción entre lenguajes, puede pensarse en lenguajes de cualquier tipo pero podemos concentrarnos en el caso de intentar traductores eficientes, en tiempo y precisión, entre lenguajes que los humanos usamos diariamente. Supongamos el caso de traducción entre Español e Inglés. Las utilidades de contar con tal tipo de herramientas es clara pero en cualquier situación se necesitará contar con un modo de determinar si las palabras involucradas en el texto de entrada al algoritmo traductor son aceptables en tal lenguaje.

Para ampliar el horizonte de utilidades es útil que nos detengamos en ciertos aspectos básicos de la teoría considerada hasta ahora. Una serie de aplicaciones muy interesantes aparece cuando estamos dispuestos a analizar con mas cuidado las hipótesis iniciales. Retornemos por un momento a las definición inicial de símbolo. Dicho concepto requiere que sea una *entidad indivisible*. Esto parece imponer un escollo en la capacidad de representar problemas o al menos en la practicidad de usar estos dispositivos que basan su funcionamiento en el procesamiento de un símbolo por vez. Pero esto puede cambiar si se interpreta dicha definición desde otra perspectiva. Basta con cambiar la interpretación de *símbolo indivisible* en la definición de alfabeto por *símbolo con significado unívoco*. Naturalmente el pedido de tener un significado unívoco se refiere a un determinado contexto de interpretación practica y concreta. No podemos esperar que para toda persona una secuencia de símbolos tenga una interpretación exactamente igual que para las demás. Pero en determinado contexto de trabajo se pueden seleccionar determinadas cadenas y asignarles un significado importante y unívoco en dicho proceso. De esta manera tales cadenas representan un elemento que quizás puedan ser descompuestas en conceptos mas primitivos pero para tal contexto o problema no interesa y a los fines practicos se

las considera como un concepto básico e indivisible. Esta es la razón por la cual en algunos textos se encontrarán autómatas cuyas transiciones esten etiquetadas con elementos mas complejos que letras o símbolos atómicos. Aquí pueden hacerse algunas observaciones, por una parte dicha extension es innecesaria ya que uno puede seguir usando solo símbolos y asociarles a ellos a través de un mapeo los significado complejo que uno desee permaneciendo fiel a la presentación teórica realizada. Otro modo de compatibilizar dicha necesidad con la teoría desarrollada precedentemente es considerar que toda cadena que denota un concepto complejo puede ser reconocida por una parte del autómata exclusivamente destinada para tal fin, como lo discutido en relación al reconocimiento de la palabra *program*. Por otra parte, es innegable que relajar tal pedido permitiendo elementos mas complejos en las transiciones, siempre y cuando se los use responsablemente, introducen comodidad en el uso de los autómatas. De allí que quizas el lector encuentre en diversos tipos de artículos tecnicos el uso de un lenguaje extendido para etiquetar las transiciones. En tal caso, en forma explícita o implícita, se estará considerando algunas de las razones recién explicadas para su adopción.

Aprovechando esta extensión en la notación podemos representar a través de un autómata finito cualquier sistema que pase por diversos estados como resultado de la ocurrencia de diversos eventos. Sigue la descripción de una serie de casos concretos destinados a ilustrar los parrafos anteriores. Por ejemplo, el comportamiento de un sistema operativo puede ser descripto como un sistema que continuamente atraviesa por una secuencia finita de estados en respuesta a los comandos con los que un usuario interactúa con el. Un lavarropas automático pasa por una secuencia finita de estados mientras está operando y esto a su vez depende del programa inicial seleccionado. Puede hacerse un analisis similar respecto de otros electrodomesticos y artefactos de funcionamiento automático o semi-automático. Por ejemplo, las máquinas expendedoras de gaseosas y golosinas pasan por estados intermedios a medida que el interesado va introduciendo monedas llegando al estado final luego que la persona pulsa una tecla para pedir un determinado artículo, siempre y cuando se haya introducido dinero suficiente. En caso contrario la maquina pasara a algun estado en el que informa que el dinero es insuficiente y dara nuevas oportunidades. Una maquina que entrega boletos en el subte tambien pasara por distintos estados a medida que se le introduce dinero culminando, si todos los pasos son realizados apropiadamente, con la entrega del boleto. El sistema de cursado de una materia consiste de una secuencia finita de estados alternativos por los que se va pasando de acuerdo al rendimiento logrado en cada evaluación.

El lector es invitado a intentar la representación de los casos antes descriptos para verificar que su funcionamiento general puede ser descripto con un autómata finito tal como es postulado en el parrafo precedente. Además, podrá comprobarse tambien que la secuencia de símbolos que representan las distintas opciones a ser procesadas por los respectivos autómatas podrán ser descriptas a través de expresiones regulares.

1.9 Referencias

El lector puede encontrar más material de estudio acerca de lo temas tratados en estos capítulos en los textos de [AU67], [HU69], [Yeh76] [DDQ78], [HU79], [LP81], [Woo87], [MG87] y [Coh91].